01_leitura_gravacao_arquivos

February 27, 2026

1 Leitura e gravação de arquivos CSV

Fundamentos de iterações via arquivos CSV, separados por vírgula, para criar dicionários e organizar estatísticas descritivas.

Leitura dos dados para dicionários

[ ]:
```
# Primeiro vamos importar o móduloCSV, o qual nos ajudará a ler nosso arquivo␣
↪CSV.
import csv

# Estabelecer a precisão de mostrar 2 casas decimais.
%precision 2

# Ler o mpg.csv usando o csv.DictReader
with open('datasets/mpg.csv') as arquivoCSV:
    # Convertendo em uma lista de dicionários
    lista_dicionarios = list(csv.DictReader(arquivoCSV))

    # OBS: O csv.DictReader considera que a primeira linha do CSV é cabeçalho,␣
↪e que o delimitador sempre é a ','. Isso são parâmetros opcionais, então se␣
↪não existir cabeçalho, precisa informar quais são as colunas, e se for outro␣
↪delimitador, precisa informar qual é.

    """
    Exemplo:

    campos = ["id", "manufacturer", "model", "displ", "year", "cyl"]
    lista_dicionarios = list(csv.DictReader(arquivoCSV, fieldnames = campos,␣
↪delimiter=';'))
    ou
    lista_dicionarios = list(csv.DictReader(arquivoCSV, fieldnames = ("tal",␣
↪"tal"...), delimiter=';'))
    """

# Agora vamos ver os três primeiros elementos de nossa lista
lista_dicionarios[:3]
```

[ ]:
```
[{'': '1',
  'manufacturer': 'audi',
  'model': 'a4',
  'displ': '1.8',
  'year': '1999',
  'cyl': '4',
  'trans': 'auto(l5)',
  'drv': 'f',
  'cty': '18',
  'hwy': '29',
  'fl': 'p',
  'class': 'compact'},
 {'': '2',
  'manufacturer': 'audi',
  'model': 'a4',
  'displ': '1.8',
  'year': '1999',
  'cyl': '4',
  'trans': 'manual(m5)',
  'drv': 'f',
  'cty': '21',
  'hwy': '29',
  'fl': 'p',
  'class': 'compact'},
 {'': '3',
  'manufacturer': 'audi',
  'model': 'a4',
  'displ': '2',
  'year': '2008',
  'cyl': '4',
  'trans': 'manual(m6)',
  'drv': 'f',
  'cty': '20',
  'hwy': '31',
  'fl': 'p',
  'class': 'compact'}]
```

[2]:
```
# Cada carro representa um dicionário
len(lista_dicionarios)
```

[2]:
```
234
```

[3]:
```
# Podemos ver o nome das colunas do CSV usando o método key
lista_dicionarios[0].keys() # Aqui vemos o nome de cada chave do dicionário
```

[3]:
```
dict_keys(['', 'manufacturer', 'model', 'displ', 'year', 'cyl', 'trans', 'drv',
'cty', 'hwy', 'fl', 'class'])
```

Cálculo separado por coluna

[4]:
```
# SE quisermos encontrar a média do consumo na cidade em milhas por galão, para␣
↪todos os carros do arquivo CSV, vamos somar todos os valores da coluna "cty"␣
↪(city miles per gallon)

consumo_medio = sum(float(dicionario['cty']) for dicionario in␣
↪lista_dicionarios) / len(lista_dicionarios)

consumo_medio
```

[4]: 16.86

[5]:
```
# Consumo na estrada

consumo_medio_estrada = sum(float(dicionario['hwy']) for dicionario in␣
↪lista_dicionarios) / len(lista_dicionarios)

consumo_medio_estrada
```

[5]: 23.44

Consumo médio na cidade agrupado por cilindros

[ ]:
```
# Queremos saber o consumo médio na cidade agrupado pelo núero de cilindros que␣
↪um carro tem
cilindros = set(dicionario['cyl'] for dicionario in lista_dicionarios)

# POR QUE UM SET (CONJUNTO)?
# R: Para eliminar as repetições
cilindros
```

[ ]: {'4', '5', '6', '8'}

[ ]:
```
# Lista para armazenar os cálculos
consumo_medio_cilindros = []

# Soma o consumo por cilindro
for cilindro in cilindros:
    soma = 0
    contador_cilindro = 0 # Conta quantos cilindros daquele tipo foram somados

    for dicionario in lista_dicionarios:
        if dicionario['cyl'] == cilindro:
            soma += float(dicionario['cty'])
            contador_cilindro += 1

    consumo_medio_cilindros.append((cilindro, soma / contador_cilindro))
```

```
# Ordena crescente
consumo_medio_cilindros.sort(key = lambda x: x[0])
consumo_medio_cilindros
```

[ ]: [('4', 21.01), ('5', 20.50), ('6', 16.22), ('8', 12.57)]

Consumo médio da cidade para as diferentes classes de veículos

[15]:
```
# Set para armazenar as diferentes classes de veículos
classes = set(dicionario['class'] for dicionario in lista_dicionarios)
classes
```

[15]: {'2seater', 'compact', 'midsize', 'minivan', 'pickup', 'subcompact', 'suv'}

[20]:
```
# Lista para armazenar o consumo para os diferentes tipos de classes
consumo_medio_classes = []

for classe in classes:
    soma = 0
    contador = 0 # Armazena a quantidade de valores da mesma classe que já␣
 ↪foram somados

    for dicionario in lista_dicionarios:
        if dicionario['class'] == classe:
            soma += float(dicionario['cty'])
            contador += 1

    carro = {'classe': classe, 'consumo_medio': soma / contador}
    consumo_medio_classes.append(carro)

consumo_medio_classes.sort(key = lambda x: x['consumo_medio'])
consumo_medio_classes
```

[20]:
```
[{'classe': 'pickup', 'consumo_medio': 13.00},
 {'classe': 'suv', 'consumo_medio': 13.50},
 {'classe': '2seater', 'consumo_medio': 15.40},
 {'classe': 'minivan', 'consumo_medio': 15.82},
 {'classe': 'midsize', 'consumo_medio': 18.76},
 {'classe': 'compact', 'consumo_medio': 20.13},
 {'classe': 'subcompact', 'consumo_medio': 20.37}]
```

02_datas_horarios

February 27, 2026

1 Datas e horátios do Python

[2]:
```
# Bibliotecas
import datetime as dt
import time as tm
```

[4]:
```
# Segundos que se passaram desde 01/01/1970
sec = tm.time()

"""
Por que existe um "tempo de início" (Epoch)?

A principal razão é padronizar o tempo. Em vez de lidar com diferentes formatos␣
↪de data, fusos horários, anos bissextos, etc., os sistemas operacionais como␣
↪o Unix precisaram de uma forma simples e prática para armazenar o tempo. E␣
↪uma forma muito eficiente de fazer isso é contar os segundos desde um ponto␣
↪fixo no tempo.

Esse ponto fixo, chamado de "Epoch", serve como o referencial universal para␣
↪todos os sistemas de computação. Ou seja, a partir desse momento, é possível␣
↪contar de forma linear e contínua o tempo que passou, o que facilita o␣
↪cálculo de duração de eventos, timestamps, e até mesmo a sincronização de␣
↪sistemas.

Utilidade prática:
Simplificação de cálculos: Para fazer cálculos de tempo em sistemas␣
↪computacionais, ter um ponto fixo (Epoch) torna mais fácil calcular duração␣
↪(como quanto tempo se passou entre dois eventos) sem ter que lidar com␣
↪complicações de datas e fusos horários.

Sincronização de sistemas: Em sistemas distribuídos, como servidores ou␣
↪dispositivos em rede, a sincronização de relógios entre máquinas é muito␣
↪mais fácil quando todas as máquinas usam o Unix Epoch como referência.

Armazenamento e manipulação de tempo: Usar um número contínuo (como o número de␣
↪segundos desde 1970) é mais eficiente para armazenar e comparar tempos, em␣
↪vez de usar datas complexas.
```

```
"""

print(sec)
```

```
1769801035.3930187
```

[15]:
```
# Pegando a hora

# Primeira forma:
data_hoje1 = dt.date.today()
"""
Essa linha cria um objeto date que contém apenas a data (ano, mês, dia). Ou␣
↪seja, não inclui a hora, minuto, segundo.
"""

# Segunda forma:
data_hoje2 = dt.datetime.fromtimestamp(tm.time())
"""
Essa linha usa o número de segundos desde 1970 (Unix Epoch) e converte esse␣
↪valor para um objeto datetime que contém tanto a data quanto a hora (ano,␣
↪mês, dia, hora, minuto, segundo, etc.).
"""


print(data_hoje1)
print(data_hoje2)
```

```
2026-01-30
2026-01-30 16:26:03.017419
```

[16]:
```
# Pegando apenas o mês, ano, dia, segundos
data_hoje1.day, data_hoje2.day, data_hoje2.second
```

[16]: (30, 30, 3)

[19]:
```
# Criando um intervalo de tempo, por exemplo em dias
delta = dt.timedelta(days = 100) # Intervalo de 100 dias

# Exemplo de como usar para somar com uma data:
data_futura = dt.datetime.now() + delta  # Soma 100 dias à data atual
print(data_futura)  # Exibe a data futura (100 dias após hoje)
```

```
2026-05-10 16:29:28.130238
```

03_funcao_map

February 27, 2026

1 Função map() do Python

[22]:
```
# A função map() no Python é usada para aplicar uma função a todos os itens de␣
↪um iterável (como uma lista, tupla, etc.) e retorna um novo iterável␣
↪(geralmente um map object) com os resultados.

# Imagine que temos duas listas de números, talvez preços de duas lojas␣
↪diferentes para os mesmos produtos, e queremos encontrar o mínimo que␣
↪teríamos que pagar se comprassemos o item mais barato entre as duas lojas.
# O map faz a comparação em uma única instrução.

loja1 = [10.00, 11.00, 12.34, 2.34]
loja2 = [9.00, 11.10, 12.34, 2.01]

mais_barato = map(min, loja1, loja2)
mais_barato
```

[22]:
```
<map at 0x7f6cf8534a60>
```

Percebe-se que o que é retornado é um valor de referência estranho, ao invés da lista de itens que estávamos esperando.

Isso é chamado de avaliação preguiçosa (lazy evaluation) –> O retorno é um objeto tipo map para economizar memória, o que significa que ele não armazena os valores na memória enquanto não for necessário. Isso nos permite ter um gerenciamento de memória muito eficiente.

Como funciona o map object: - Preguiçoso: O map object não armazena todos os resultados de uma vez. Ele calcula o valor de cada item sob demanda, ou seja, quando você realmente precisar dele. - Gerador: Internamente, o map() funciona como um gerador, ou seja, ele gera os valores à medida que são solicitados, sem ocupar memória para armazenar todos de uma vez.

Nesse caso, temos duas opções para acessar os valores 1° Caso) Transformar o map object em uma lista:

[10]:
```
# Transformar o map object em uma lista:
mais_barato_lista = list(mais_barato)
mais_barato_lista
```

```
# E nesse caso perderemos a vantagem de economia de memória
```

[10]: [9.0, 11.0, 12.34, 2.01]

2° Caso) Iterar diretamento sobre o map object, usando um for loop ou até a função next() para obter o valor desejado:

[20]:
```
# Iterando e acessando o valor do produto 3 (índice 2)
# Com for:
i = 0
for valor in mais_barato:
    # Verificando se é o produto dois
    if i == 2:
        print(valor)
        break

    i += 1
```

12.34

[23]:
```
# Com next
next(mais_barato) # Produto 1
next(mais_barato) # Produto 2
produto_3 = next(mais_barato) # Produto 3

print(produto_3)
```

12.34

04_lambda

February 27, 2026

1 Função Lambda

Forma do Python para criar funções anônimas, isto é, função sem nome. A intenção é que elas sejam simples ou curtas e que sejam fáceis para escrever em uma linha ao invés de ter o trabalho de criar uma função com o nome.

Usos comuns para lambda:

- Funções rápidas e simples: As funções lambda são comumente usadas quando você precisa de uma função temporária para uma operação simples, sem a necessidade de definir uma função completa.
- Funções como argumentos: lambda é frequentemente usada em funções de ordem superior como map(), filter(), e sorted(), que aceitam funções como argumentos.

[1]:
```
# Você declara a função lambda com a palavra lambda, seguida por uma lista de␣
↪argumentos, seguidos por ":", e então uma única expressão e isto é a chave.
# O valor da expressão é retornado na execução da lambda.

minha_funcao = lambda a, b, c: a + b + c
minha_funcao(1, 2, 3)
```

[1]: 6

[ ]:
```
numeros = [1, 2, 3, 4]
quadrados = map(lambda x: x ** 2, numeros)
print(list(quadrados))  # Exibe: [1, 4, 9, 16]
```

Uma forma compacta (tende a ser mais rápida):

[2]:
```
lista = [numero for numero in range(1000) if numero % 2 == 0]
lista
```

[2]:
```
[0,
 2,
 4,
 6,
 8,
 10,
 12,
```

```
954,
956,
958,
960,
962,
964,
966,
968,
970,
972,
974,
976,
978,
980,
982,
984,
986,
988,
990,
992,
994,
996,
998]
```

[4]:
```python
print(range(1000))
```

```
range(0, 1000)
```

05_criacao_matriz

February 27, 2026

1 Criação de Matriz

NumPy é o pacotefundamental para computação númerica com Python. Ele fornece maneiras poderosas de criar, armazenar e manipular dados, o que o torna capaz de se integrar de forma perfeita e rápida a uma ampla variedade de bancos de dados e formatos de dados. Essa também é a base sobre a qual o Pandas se baseia, que é um pacote centrado em dados de alto desempenho.

Vamos criar matrizes com determinados tipos de dados, manipular matrizes, selecionar elementos de matrizes e carregar conjuntos de dados em matrizes.

Essas funções são úteis para manipular dados e entender as funcionalidades de outros pacotes de dados comuns do Python.

[1]:
```
# A abreviatura comum do NumPy é "np"

# Bibliotecas
import numpy as np
import math
```

Criação de Array

[ ]:
```
# As matrizes são exibidas como uma lista ou lista de listas e também podem ser␣
↪criadas por meio de listas.
# Ao criar um vetor, passamos para ele uma lista como argumento em uma lista␣
↪numérica.

a = np.array([1, 2, 3])
print(a)

# Podemos imprimir o número de dimensões de uma lista usando:
print(a.ndim)
```

```
[1 2 3]
1
```

[18]:
```
# Se passarmos uma lista de listas para um vetor numpy, criaremos um vetor␣
↪multidimensional, por exemplo, uma matriz.
b = np.array([[1, 2, 3], [4, 5, 6]])
print(b)
```

```
print(b.ndim)
```

```
[[1 2 3]
 [4 5 6]]
2
```

[9]:
```
# Podemos imprimir o comprimento de cada dimensão chamando o atributo shape,␣
↪que retorna uma tupla:
b.shape
```

[9]: (2, 3)

[ ]:
```
# Também podemos verificar o tipo de itens na matriz, então um dtype.
a.dtype
```

[ ]: dtype('int64')

[21]:
```
# Além dos números inteiros, os floats também são aceitos nas matrizes numpy
c = np.array([2.2, 5, 1.1])
c.dtype.name
```

[21]: 'float64'

[22]:
```
# Vamos ver os dados em nosso vetor
c
```

[22]: array([2.2, 5. , 1.1])

Perceba que o numpy converte automaticamente números inteiros, como 5, até pontos flutuantes. Como não há perda de precisão, o numpy tentará fornecer o melhor formato de tipo de dados possível para manter seus tipos de dados homogêneos, o que significa que eles são todos iguais na vetor.

OBS: Por isso quando colocamos uma str no vetor, TODOS os outros dados terão o tipo de dado mais geral entre os vetores.

[ ]:
```
# As vezes, sabemos a forma de um vetor/matriz que queremos criar, mas não o␣
↪que queremos que esteja nela.
# O numpy oferece várias funções para criar matrizes com espaços reservados␣
↪iniciais, como zeros ou uns.
# Vamos criar duas matrizes, ambas com a mesma forma, mas com valores de␣
↪preenchimento diferentes.

d = np.zeros((2, 3), int)
print(d)

e = np.ones((2, 3))
print(e)
```

```
[[0 0 0]
 [0 0 0]]
[[1. 1. 1.]
 [1. 1. 1.]]
```

[ ]:
```python
# Também podemos gerar uma matriz com números aleatórios.
f1 = np.random.rand(2, 3)

# Se fosse inteiros:
f2 = np.random.randint(0, 10, (2, 3))

print(f1)
print('\n', f2)
```

```
[[0.37012286 0.62993629 0.23023623]
 [0.14678558 0.44380166 0.9075451 ]]

 [[2 3 3]
 [5 9 6]]
```

[ ]:
```python
# Também podemos criar uma sequência de números em uma matriz com a função de␣
↪arranjo. O primeiro argumento é o limite inicial, o segundo argumento é o␣
↪limite final - 1 e o terceiro argumento é a diferença entre cada número␣
↪consecutivo.

# Vamos criar um vetor de todos os números pares de 10 até 49.
g = np.arange(10, 50, 2)
print(g)
```

```
[10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50]
```

[ ]:
```python
# Vamos dar uma olhada em G.Se quisermos gerar uma sequência de pontos␣
↪flutuantes, usamos a função linspace().
# O terceiro argumento não é a diferença entre dois números, mas é o número␣
↪total de itens que você deseja gerar.
h = np.linspace(0, 2, 15) # 15 números entre 0 a 2, incluindo o 0 e o 2.
print(h)
```

```
[0.         0.14285714 0.28571429 0.42857143 0.57142857 0.71428571
 0.85714286 1.         1.14285714 1.28571429 1.42857143 1.57142857
 1.71428571 1.85714286 2.        ]
```

06_matriz_manipulacao

February 27, 2026

1 Manipulação com matriz

Podemos fazer muitas coisas em matrizes, como manipulação matemática, adição, subtração, expoentes quadrados e matrizes booleanas, que são valores binários. Também podemos fazer manipulação de matrizes, como transposição de produto, inversa e assim por diante.

[1]:
```
import numpy as np
```

Operações aritméticas

[ ]:
```
# Operadores aritméticos em vetores se aplicam a elementos.

a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

c = a - b
print(c)

c = a * b
print(c)

# Com a manipulação aritimética, podemos converter os dados atuais do jeito que␣
↪queremos que sejam.
# Os operadores aritméticos em vetores do NumPy permitem realizar operações␣
↪element-wise (ou seja, operação realizada individualmente em cada elemento␣
↪do vetor).
# ATENÇÃO: Se for operação entre dois vetores, as dimensoes dos vetores␣
↪precisam ser iguais para funcionar. Com matrizes, isso muda um pouco.
```

```
[ 9 18 27 36]
[ 10  40  90 160]
```

[48]:
```
# Precisão de duas casas decimais
%precision 2

# Vamos transformar farenheit em celcius
farenheit = np.random.randint(-50, 100, 15)
farenheit.sort()
```

```
celsius = (farenheit - 31) * (5/9)
print(celsius)
```

```
[-43.33 -30.   -28.89 -22.22 -13.89 -13.33  -7.78  -1.11   4.44   5.56
  13.33  15.    16.11  32.78  35.56]
```

Matriz booleana

[ ]:
```
# Outra manipulação importante é a matriz booleana. Podemos aplicar o operador␣
↪em uma matriz, e uma matriz booleana será retornada para qualquer elemento␣
↪no original com True sendo emitido se atender à condição.

# Por exemplo, se quisermos obter uma matriz booleana para verificar os graus␣
↪Celsius maiores que -20 graus:

celsius > -20

# Isso retorna uma matriz booleana
```

[ ]:
```
array([False, False,  True,  True,  True,  True,  True,  True,  True,
       True,  True,  True,  True,  True,  True])
```

[22]:
```
# Podemos usar o operador de módulo para verificar números e matrizes para ver␣
↪se eles são pares.

numeros = np.random.randint(0, 100, 30)
print(numeros)

numeros % 2 == 0
```

```
[31  6 72 92  9 40 31 65 80 34 55 12 35 86 38 59 53 16 34 15 48 20 15 55
 58 23 77  0 93 51]
```

[22]:
```
array([False,  True,  True,  True, False,  True, False, False,  True,
       True, False,  True, False,  True,  True, False, False,  True,
       True, False,  True,  True, False, False,  True, False, False,
       True, False, False])
```

Produto Matricial

[44]:
```
# Alem da manipulação por elementos, é importante saber que o numpy suporta a␣
↪manipulação de matrizes.

# Vamos analisar o produto matricial
# Se quisermos fazer um produto por elemento, usamos o sinal de asterisco "*",␣
↪igual fizemos entre dois vetores:
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])

C = A * B
print(C)

# Nesse caso, cada um dos elementos de uma matriz foi multiplicado com os␣
↪elementos da outra matriz na mesma posição

# MAS QUEREMOS UM PRODUTO ENTRE MATRIZES, então faremos o sinal de ARROBA "@"
C = A @ B
print('\n', C)

# ATENÇÃO: Se tivermos tipos diferentes, o produto matricial resultante terá o␣
↪tipo de dado mais geral dos dois tipos
```

```
[[ 5 12]
 [21 32]]

 [[19 22]
 [43 50]]
```

[ ]:
```
# Vamos criar uma matriz de inteiros e uma matriz de floats
matriz_inteira = np.random.randint(0, 10, (2, 3))
matriz_float = np.random.rand(2, 3) * 10

# Verificando tipos
print(matriz_inteira.dtype)
print(matriz_float.dtype)
```

```
int64
float64
```

[ ]:
```
# Ao fazer uma operação em uma matriz de inteiros com uma matriz de floats, o␣
↪resultado é o tipo mais geral, nesse caso, float.
matriz_resultado = matriz_inteira + matriz_float
print(matriz_resultado)
print(matriz_resultado.dtype)
```

```
[[ 7.61  4.67 17.69]
 [10.28 16.08  5.04]]
float64
```

Funções de agregação

[52]:
```
# Os Arrays da numpy possuem funções de agragação interessantes, como sum(),␣
↪max(), min() e mean()
print(matriz_resultado.sum())
print(matriz_resultado.max())
```

```
print(matriz_resultado.min())
print(matriz_resultado.mean())
```

```
61.35754965134735
17.686746459083967
4.667074152637429
10.226258275224557
```

[111]:
```
# Para vetores bidimensionais (matrizes), podemos fazer a mesma coisa para cada␣
↪linha ou coluna
matriz = np.random.randint(1, 16, (3, 5))

# Outra forma de criar uma matriz de inteiros:
matriz1 = np.arange(0, 15).reshape(3, 5)
print(matriz1)

print('\n', matriz)

# Somando itens da linha
print(sum(m for m in matriz[0]))


# Somando itens da coluna
print(sum(m[0] for m in matriz))
```

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]

 [[12  6  4  4 10]
 [ 2  5 10 13 10]
 [ 7 14 15  8  9]]
36
21
```

Abstração em matrizes Muitas vezes vemos um vetor bidimensional composto por linhas e colunas, mas você pode pensar nessas matrizes como listas gigantes ordenadas de números. A forma da matriz, o número de linhas e colunas, é uma abstração que temos para um propósito específico. Na verdade, é exatamente assim que as imagens básicas são armazenadas em ambientes de computador.

Vamos dar uma olhada em um exemplo de como o numpy funciona com imagens:

[78]:
```
# Para o uso de imagens, vamos usar a Python Imagin Library (PIL) e uma função␣
↪para exibir imagens aqui.
# !pip install Pillow

from PIL import Image
```

```python
from IPython.display import display

# Vamos ver a imagem
img = Image.open('img/chris.tiff')
display(img)
```

[82]:
```python
# Agora, podemos converter essa imagem PIL em uma matriz numpy
matriz_img = np.array(img)
print(matriz_img.shape)
matriz_img
```

```
(200, 200)
```

[82]:
```
array([[118, 117, 118, …, 103, 107, 110],
       [113, 113, 113, …, 100, 103, 106],
       [108, 108, 107, …,  95,  98, 102],
       …,
       [177, 181, 182, …, 193, 198, 192],
       [178, 182, 183, …, 193, 201, 189],
       [178, 182, 184, …, 193, 201, 187]], shape=(200, 200), dtype=uint8)
```

Perceba que todos os valores são do tipo "uint8", o que significa que (uint) são números inteiros sem sinal, e o (8) indica a quantidade de bits de cada valor. Então, isso significa que cada valor da matriz_img vai de 0 até 2^8 - 1 (de 0 a 255).

Para imagens em preto e branco, o preto é armazenado com valor zero (0) e o branco é armazenado com valor máximo (255).

[83]:
```python
# Vamos inverter as cores dessa imagem usando uma matriz numpy
```

```
# Vamos criar uma matriz com a mesma forma:
inverter = np.full(matriz_img.shape, 255)
print(inverter)
```

```
[[255 255 255 … 255 255 255]
 [255 255 255 … 255 255 255]
 [255 255 255 … 255 255 255]
 …
 [255 255 255 … 255 255 255]
 [255 255 255 … 255 255 255]
 [255 255 255 … 255 255 255]]
```

[102]:
```
# Agora vamos subtrair isso da imagem

matriz_modificada = (matriz_img - inverter) * -1

# Vamos dizer ao numpy que defina o valor do tipo de dados corretamente
matriz_modificada = matriz_modificada.astype(np.uint8)
matriz_modificada
```

[102]:
```
array([[137, 138, 137, …, 152, 148, 145],
       [142, 142, 142, …, 155, 152, 149],
       [147, 147, 148, …, 160, 157, 153],
       …,
       [ 78,  74,  73, …,  62,  57,  63],
       [ 77,  73,  72, …,  62,  54,  66],
       [ 77,  73,  71, …,  62,  54,  68]], shape=(200, 200), dtype=uint8)
```

[104]:
```
# Vamos exibir essa nova matriz. Podemos fazer isso usando a função fromarray()␣
↪e a PIL para converter a matriz em um objeto que poderá ser renderizado.
↪matriz_modificada

display(Image.fromarray(matriz_modificada))
```

Perceba que o que era branco ficou preto e o que era preto ficou branco.

[114]:
```
# Como isso poderia ser pensado como uma matriz gigante de bytes e essa forma␣
↪era uma abastração, poderíamos decidir remodelar a matriz e ainda tentar␣
↪renderizá-la.
# O PIL está interpretando as linhas individuas como linhas, então podemos␣
↪alterar o número de linhas e colunas se quisermos.

reshaped = np.reshape(matriz_modificada, (100, 400))
# ATENÇÃO: Estamos alterando a largura e a altura, mas teremos a mesma␣
↪quantidade de células (antes: 200x200 = 40000, agora: 100x400 = 40000␣
↪células)
print(reshaped.shape)

display(Image.fromarray(reshaped))
```

```
(100, 400)
```

Perceba que ao remodelar a matriz para ter apenas 100 linhas de altura, mas 400 colunas, basica-

mente dobramos a imagem, pegando todas as outras linhas e empilhando-as em largura.

A ideia não é mexer com imagens, mas mostrar que as matrizes são abstrações sobre os dados.

ATENÇÃO: Como cientista de dados, o nosso papel é entender o que os dados significam, seu contexto em uma coleção e transformá-los em uma representação diferente a ser usada para criar sentido.

07_indexacao_fatiamento_iteracao

February 27, 2026

1 Indexação, fatiamento e iteração

[1]:
```
import numpy as np
```

Indexação

[2]:
```
# Vamos criar um vetor
a = np.array([1, 2, 3, 4, 5, 6, 7])
a[2]
```

[2]:
```
np.int64(3)
```

[3]:
```
# Para vetores multidimensionais, precisamos usar a indexação de matrizes␣
↪inteiras.
a = np.array([[1, 2], [3, 4], [5, 6]])
print(a)
a[1][1]
```

```
[[1 2]
 [3 4]
 [5 6]]
```

[3]:
```
np.int64(4)
```

[ ]:
```
# Podemos criar uma nova matriz, e nela vamos passar uma lista e, dessa lista,␣
↪estamos na verdade pegando nossa outra matriz e retirando os valores nos␣
↪quais estamos interessados
np.array([a[0, 0], a[1, 1], a[2, 1]])
```

[ ]:
```
array([1, 4, 6])
```

[6]:
```
# Também podemos fazer isso usando outra forma de indexação de matrizes, na␣
↪qual basicamente compacta a primeira lista e a segunda.
print(a[[0, 1, 2], [0, 1, 1]])

# Perceba que nesse caso, está pegando como se fosse pares ordenados de uma␣
↪lista e da outra
```

```
[1 4 6]
```

Indexação booleana

[7]:
```
# A indexação booleana nos permite selecionar elementos arbitrários com base␣
↪nas condições.
print(a > 5)
```

```
[[False False]
 [False False]
 [False  True]]
```

[ ]:
```
# Podemos colocar essa matriz de valores booleanos, como uma máscara booleana,␣
↪sobre a matriz original para retornar uma matriz unidimensional relacionada␣
↪aos valores reais
print(a[a > 5])

# O que está acontecendo aqui é que pegaremos o operador maior que cinco e␣
↪transmitiremos isso. Então será comparado isso em todos os elementos de "a",␣
↪criando uma nova matriz com apenas os valores que satisfazem a condição.
# Ao usar essa máscara booleana para indexar o array original, você está␣
↪filtrando os valores que atendem à condição (> 5). Ou seja, você pega os␣
↪valores da matriz onde a condição é True.
```

```
[6]
```

Fatiamento

[10]:
```
# Fatiar é uma forma de criar uma submatriz com base na matriz original. Para␣
↪vetores unidimensionais, o fatiamento funciona de forma semelhante a uma␣
↪lista.
a = np.array([0, 1, 2, 3, 4, 5, 6, 7])

# Pegando do índice 0 até o 2
print(a[:3])

# Pegando do índice 2 até o 4
print(a[2:5])
```

```
[0 1 2]
[2 3 4]
```

[48]:
```
# Para matrizes multidimensionais, funciona de forma semelhante
a = np.random.randint(0, 100, (10, 5))
print(a)
```

```
[[27  5 53 67 40]
 [71  6 95 78 12]
 [72 43 50 28 36]
```

```
 [14 18 44 61 20]
 [34 77 28 11 48]
 [26 57 73 13 30]
 [19 58 53  1 34]
 [78 88 93  7 73]
 [30 19 11 42 78]
 [ 5 89 67 48  1]]
```

[49]:
```
# Quando fazemos a[0], estamos pegando a primeira linha inteira, então se␣
↪fizermos a[:n], estaremos pegando da linha 0 até a linha 'n'
print(a[:2])

print()

# Pegando da linha 4 até a linha 10
print(a[4:10])
```

```
[[27  5 53 67 40]
 [71  6 95 78 12]]

[[34 77 28 11 48]
 [26 57 73 13 30]
 [19 58 53  1 34]
 [78 88 93  7 73]
 [30 19 11 42 78]
 [ 5 89 67 48  1]]
```

[50]:
```
# Quando fazemo a[0][0], estamos pegando o primeiro item da primeira coluna,␣
↪então se fizermos a[0][:n] estaremos fatiando na coluna, ou seja, seria da␣
↪linha 0 pegar os valores de 0 até n - 1. Por exemplo:

print(a[0][:4])
print()
print(a[4][:2])
```

```
[27  5 53 67]

[34 77]
```

[51]:
```
# Seguindo nessa lógica pensamos que ao fazer a[:4][:2] estaremos fatiando a␣
↪linha e a coluna, porém muito CUIDADO, pois NÃO ESTAREMOS FATIANDO NA COLUNA␣
↪(que seria andar na linha), o que acontece na realidade é o fatiamento do␣
↪fatiamento, por exemplo:
print(a[:4])
print()
print(a[:4][1:3])
```

```
# Nesse caso, pegamos as linhas 0 até 3, e dessas 4 linhas pegadas, fizemos um␣
↪novo fatiamento pegando da 1 até 2
```

```
[[27  5 53 67 40]
 [71  6 95 78 12]
 [72 43 50 28 36]
 [14 18 44 61 20]]

[[71  6 95 78 12]
 [72 43 50 28 36]]
```

[52]:
```
print(f'{a}\n')

# Para fazer o fatiamento de linhas e colunas ao mesmo tempo, usa-se:
print(a[:8, 1:3])

# Aqui pegamos da linha 0 até a linha 7, e dessas linhas pegamos os valores das␣
↪colunas 1 (segunda) e 2 (terceira)
```

```
[[27  5 53 67 40]
 [71  6 95 78 12]
 [72 43 50 28 36]
 [14 18 44 61 20]
 [34 77 28 11 48]
 [26 57 73 13 30]
 [19 58 53  1 34]
 [78 88 93  7 73]
 [30 19 11 42 78]
 [ 5 89 67 48  1]]

[[ 5 53]
 [ 6 95]
 [43 50]
 [18 44]
 [77 28]
 [57 73]
 [58 53]
 [88 93]]
```

É importante se atentar que modificar a submatriz também modificará a matriz original, já que os valores são passados por referência.

Exemplo:

[53]:
```
# Matriz original antes
print(a, '\n')

# sub matriz
```

```
sub_matriz = a[:2, 1:3]
print(sub_matriz, '\n')

sub_matriz[0][0] = 1000

# Matriz original depois
print(a)
```

```
[[27  5 53 67 40]
 [71  6 95 78 12]
 [72 43 50 28 36]
 [14 18 44 61 20]
 [34 77 28 11 48]
 [26 57 73 13 30]
 [19 58 53  1 34]
 [78 88 93  7 73]
 [30 19 11 42 78]
 [ 5 89 67 48  1]]

[[ 5 53]
 [ 6 95]]

[[  27 1000   53   67   40]
 [  71    6   95   78   12]
 [  72   43   50   28   36]
 [  14   18   44   61   20]
 [  34   77   28   11   48]
 [  26   57   73   13   30]
 [  19   58   53    1   34]
 [  78   88   93    7   73]
 [  30   19   11   42   78]
 [   5   89   67   48    1]]
```

Em lista, faziamos um fatiamento para copiar os valores, por exemplo:

b = a[:] Assim é uma cópia de "a" dentro de "b"

Porém aqui, para copiar, usa-se como dicionários, com .copy()

[55]:
```
# Matriz original antes
print(a, '\n')

# sub matriz
sub_matriz = a[:2, 1:3].copy()
print(sub_matriz, '\n')

sub_matriz[0][0] = 9999
print(sub_matriz, '\n')
```

```python
# Matriz original depois
print(a)
```

```
[[  27 1000   53   67   40]
 [  71    6   95   78   12]
 [  72   43   50   28   36]
 [  14   18   44   61   20]
 [  34   77   28   11   48]
 [  26   57   73   13   30]
 [  19   58   53    1   34]
 [  78   88   93    7   73]
 [  30   19   11   42   78]
 [   5   89   67   48    1]]

[[1000   53]
 [   6   95]]

[[9999   53]
 [   6   95]]

[[  27 1000   53   67   40]
 [  71    6   95   78   12]
 [  72   43   50   28   36]
 [  14   18   44   61   20]
 [  34   77   28   11   48]
 [  26   57   73   13   30]
 [  19   58   53    1   34]
 [  78   88   93    7   73]
 [  30   19   11   42   78]
 [   5   89   67   48    1]]
```

08_numpy_conjunto_dados

February 27, 2026

1 Experimentando o NumPy com conjuntos de dados

Aqui temos um conjunto de dados muito popular sobre a qualidade do vinho.

Conjunto 1: Vinhos

[141]:
```
import numpy as np
```

[142]:
```
# Para carregar um conjunto de dados no numpy podemos usar a função␣
↪genfromtxt().
# Podemos especificar o nome do arquivo de dados, o delimitador, na qual é␣
↪opcional, e o número de linhas a serem ignoradas (normalmente o cabeçalho).

# A função genfromtxt() tem um parâmetro chamado data types(opcional) para␣
↪especificar os tipos de dados para cada coluna. Caso não especifiquemos o␣
↪tipo, será o tipo mais geral.

vinhos = np.genfromtxt('datasets/winequality-red.csv', delimiter = ';',␣
↪skip_header = 1)

vinhos
```

[142]:
```
array([[ 7.4  ,  0.7  ,  0.   , …,  0.56 ,  9.4  ,  5.   ],
       [ 7.8  ,  0.88 ,  0.   , …,  0.68 ,  9.8  ,  5.   ],
       [ 7.8  ,  0.76 ,  0.04 , …,  0.65 ,  9.8  ,  5.   ],
       …,
       [ 6.3  ,  0.51 ,  0.13 , …,  0.75 , 11.   ,  6.   ],
       [ 5.9  ,  0.645,  0.12 , …,  0.71 , 10.2  ,  5.   ],
       [ 6.   ,  0.31 ,  0.47 , …,  0.66 , 11.   ,  6.   ]],
      shape=(1599, 12))
```

[143]:
```
# Vamos pegar apenas os dados da primeira coluna:
print(f'Dados da coluna 0: {vinhos[:, 0]}')
```

```
Dados da coluna 0: [7.4 7.8 7.8 … 6.3 5.9 6. ]
```

[144]:
```
# Mas vamos supor que eu gostaria que eles ainda ficassem no formato de coluna,␣
↪então fazemos um fatiamento na coluna:
```

```
print(f'Dados da coluna 0 na forma de coluna:\n {vinhos[:, 0:1]}')

# Nesse caso, preservamos a forma geral de que se trata de uma única coluna
```

```
Dados da coluna 0 na forma de coluna:
 [[7.4]
 [7.8]
 [7.8]
 …
 [6.3]
 [5.9]
 [6. ]]
```

Isso é um exemplo de como a forma dos dados é, na verdade, apenas uma abstração, que podemos sobrepor intencionalmente aos dados com os quais estamos trabalhando.

[145]:
```
# Se quisermos um intervalo de colunas em ordem, podemos fazer:
print(f'Dados das colunas 0 à 2: \n{vinhos[:, 0:3]}')
```

```
Dados das colunas 0 à 2:
[[7.4   0.7   0.   ]
 [7.8   0.88  0.   ]
 [7.8   0.76  0.04 ]
 …
 [6.3   0.51  0.13 ]
 [5.9   0.645 0.12 ]
 [6.    0.31  0.47 ]]
```

[146]:
```
# Mas e se quisermos colunas específicas, isto é, não consecutivas?
# Então passamos elas como listas para dentro do índice da matriz:
print(f'Dados das colunas 0, 2 e 4: \n{vinhos[:, [0, 2, 4]]}')
```

```
Dados das colunas 0, 2 e 4:
[[7.4   0.    0.076]
 [7.8   0.    0.098]
 [7.8   0.04  0.092]
 …
 [6.3   0.13  0.076]
 [5.9   0.12  0.075]
 [6.    0.47  0.067]]
```

[147]:
```
# Também podemos fazer um resumo básico desse conjunto de dados.

# Por exemplo, se quisermos descobrir a qualidade média do vinho tinto, podemos␣
↪selecionar a coluna de qualidade.
# Podemos fazer isso de duas maneiras, porém a mais apropriada é usar o valor␣
↪de '-1' para o índice, pois números negativos significam cortar do final de␣
↪uma lista. Em seguida, basta chamar as funções de agregação nesses dados.
```

```
# Pegando a última coluna e calculando a média
media1 = vinhos[:, -1].mean() # Melhor forma
print(media1, '\n')

# Outra forma:
media2 = sum(vinhos[:, -1]) / len(vinhos[:, -1])
print(media2, '\n')

# Ainda mais um jeito:
media3 = sum(m[len(vinhos[0]) - 1] for m in vinhos) / len(vinhos[:, 0])
print(media3)
```

```
5.6360225140712945

5.6360225140712945

5.6360225140712945
```

Conjunto 2: Admissões Vamos olhar outro conjunto de dados, agora sobre admissões em escolas de pós-graduação.

Aqui são pontos como GRE, pontuação TOEFL, classificação universitária… Tem uma chance de admissão no final. Com esse conjunto de dados, podemos fazer manipulação de dados e análise básica para inferir quais condições estão associadas a maiores chances de admissão.

[148]:
```
# Podemos especificar nomes de campos de dados usando genfromtxt() enquanto ele␣
↪carrega os dados CSV. Além disso, podemos fazer com que o numpy tente␣
↪inferir esse tipo de coluna definindo o parâmetro do dtype (tipo do dado)␣
↪como nenhum

admissao_graduacao = np.genfromtxt('datasets/Admission_Predict.csv', delimiter␣
↪= ',', skip_header = 1, names = ("Número_de_Série", "Pontuação_GRE",␣
↪"Pontuação_TOEFL", "Classificação_da_Universidade", "SOP", "LOR", "CGPA",␣
↪"Pesquisa", "Chance_de_Admissao"))

# ATENÇÃO: Os nomes não podem possuir espaços, por isso use "_"

admissao_graduacao
```

[148]:
```
array([(  1., 337., 118., 4., 4.5, 4.5, 9.65, 1., 0.92),
       (  2., 324., 107., 4., 4. , 4.5, 8.87, 1., 0.76),
       (  3., 316., 104., 3., 3. , 3.5, 8.  , 1., 0.72),
       (  4., 322., 110., 3., 3.5, 2.5, 8.67, 1., 0.8 ),
       (  5., 314., 103., 2., 2. , 3. , 8.21, 0., 0.65),
       (  6., 330., 115., 5., 4.5, 3. , 9.34, 1., 0.9 ),
       (  7., 321., 109., 3., 3. , 4. , 8.2 , 1., 0.75),
       (  8., 308., 101., 2., 3. , 4. , 7.9 , 0., 0.68),
```

```
       (385., 340., 113., 4., 5. , 5. , 9.74, 1., 0.96),
       (386., 335., 117., 5., 5. , 5. , 9.82, 1., 0.96),
       (387., 302., 101., 2., 2.5, 3.5, 7.96, 0., 0.46),
       (388., 307., 105., 2., 2. , 3.5, 8.1 , 0., 0.53),
       (389., 296.,  97., 2., 1.5, 2. , 7.8 , 0., 0.49),
       (390., 320., 108., 3., 3.5, 4. , 8.44, 1., 0.76),
       (391., 314., 102., 2., 2. , 2.5, 8.24, 0., 0.64),
       (392., 318., 106., 3., 2. , 3. , 8.65, 0., 0.71),
       (393., 326., 112., 4., 4. , 3.5, 9.12, 1., 0.84),
       (394., 317., 104., 2., 3. , 3. , 8.76, 0., 0.77),
       (395., 329., 111., 4., 4.5, 4. , 9.23, 1., 0.89),
       (396., 324., 110., 3., 3.5, 3.5, 9.04, 1., 0.82),
       (397., 325., 107., 3., 3. , 3.5, 9.11, 1., 0.84),
       (398., 330., 116., 4., 5. , 4.5, 9.45, 1., 0.91),
       (399., 312., 103., 3., 3.5, 4. , 8.78, 0., 0.67),
       (400., 333., 117., 4., 5. , 4. , 9.66, 1., 0.95)],
      dtype=[('Número_de_Série', '<f8'), ('Pontuação_GRE', '<f8'),
('Pontuação_TOEFL', '<f8'), ('Classificação_da_Universidade', '<f8'), ('SOP',
'<f8'), ('LOR', '<f8'), ('CGPA', '<f8'), ('Pesquisa', '<f8'),
('Chance_de_Admissao', '<f8')])
```

[149]:
```
# Perceba que a matriz possui uma dimensão, com tuplas de tantas colunas
admissao_graduacao.shape
```

[149]:
```
(400,)
```

[150]:
```
# Podemos recuperar uma coluna da matriz usando o nome da coluna. Por exemplo,␣
↪vamos pegar a coluna CGPA e somente os primeiros cinco valores.

# Vamos pegar apenas a coluna CGPA e mostrar as primeiras 5 linhas (5 valores)
admissao_graduacao['CGPA'][:5]
```

[150]:
```
array([9.65, 8.87, 8.  , 8.67, 8.21])
```

[151]:
```
# Como o GPA no conjunto de dados varia de 1 a 10, e nos EUA é mais comum usar␣
↪uma escala de quatro. Uma tarefa comum pode ser converter o GPA dividindo␣
↪por 10 e depois multiplicando por quatro

admissao_graduacao['CGPA'] = (admissao_graduacao['CGPA'] / 10) * 4

# Vamos analisar 20 valores:
admissao_graduacao['CGPA'][:20]
```

[151]:
```
array([3.86 , 3.548, 3.2  , 3.468, 3.284, 3.736, 3.28 , 3.16 , 3.2  ,
       3.44 , 3.36 , 3.6  , 3.64 , 3.2  , 3.28 , 3.32 , 3.48 , 3.2  ,
       3.52 , 3.4  ])
```

Acabamos de realizar uma normalização para valores até 4

[152]:
```
# Lembrando da máscara booleana, podemos usar isso para descobrir quantos␣
↪estudante tiveram experiência em pesquisa.

# Vamos criar a máscara booleana e passar para o operador de indexação de␣
↪matriz.
# Perceba no CSV que Reaserch é um valor de 1 e 0.
qnts_estudantes = len(admissao_graduacao[admissao_graduacao['Pesquisa'] == 1])
print(qnts_estudantes)
```

```
219
```

[153]:
```
# Como temos a chance de admissão no campo de dados, que varia de 0 a 1,␣
↪podemos tentar ver se os alunos com alta chance de admissão (> 0.8) tem␣
↪pontuação GRE mais altas do que aqueles com menor chance de admissão (< 0.4).
↪

# Vamos usar o mascaramento booleano para retirar apenas os alunos nos quais␣
↪estamos interessados com base em suas chances de admissão.
print(admissao_graduacao[admissao_graduacao['Chance_de_Admissao'] > 0.
↪8]['Pontuação_GRE'].mean())
print(admissao_graduacao[admissao_graduacao['Chance_de_Admissao'] < 0.
↪4]['Pontuação_GRE'].mean())
```

```
328.7350427350427
302.2857142857143
```

Quando fazemos o mascaramento booleano, ainda temos uma matriz com tuplas, e o numpy mantém abaixo dela uma lista das colunas que especificamos e seus nomes e índices.

[154]:
```
# Vamos olhar isso:
admissao_graduacao[admissao_graduacao['Chance_de_Admissao'] > 0.8]
```

[154]:
```
array([(  1., 337., 118., 4., 4.5, 4.5, 3.86 , 1., 0.92),
       (  6., 330., 115., 5., 4.5, 3. , 3.736, 1., 0.9 ),
       ( 12., 327., 111., 4., 4. , 4.5, 3.6  , 1., 0.84),
       ( 23., 328., 116., 5., 5. , 5. , 3.8  , 1., 0.94),
       ( 24., 334., 119., 5., 5. , 4.5, 3.88 , 1., 0.95),
       ( 25., 336., 119., 5., 4. , 3.5, 3.92 , 1., 0.97),
       ( 26., 340., 120., 5., 4.5, 4.5, 3.84 , 1., 0.94),
       ( 33., 338., 118., 4., 3. , 4.5, 3.76 , 1., 0.91),
       ( 34., 340., 114., 5., 4. , 4. , 3.84 , 1., 0.9 ),
       ( 35., 331., 112., 5., 4. , 5. , 3.92 , 1., 0.94),
       ( 36., 320., 110., 5., 5. , 5. , 3.68 , 1., 0.88),
       ( 44., 332., 117., 4., 4.5, 4. , 3.64 , 0., 0.87),
       ( 45., 326., 113., 5., 4.5, 4. , 3.76 , 1., 0.91),
       ( 46., 322., 110., 5., 5. , 4. , 3.64 , 1., 0.88),
```

```
       (383., 324., 110., 4., 4.5, 4. , 3.66 , 1., 0.82),
       (385., 340., 113., 4., 5. , 5. , 3.896, 1., 0.96),
       (386., 335., 117., 5., 5. , 5. , 3.928, 1., 0.96),
       (393., 326., 112., 4., 4. , 3.5, 3.648, 1., 0.84),
       (395., 329., 111., 4., 4.5, 4. , 3.692, 1., 0.89),
       (396., 324., 110., 3., 3.5, 3.5, 3.616, 1., 0.82),
       (397., 325., 107., 3., 3. , 3.5, 3.644, 1., 0.84),
       (398., 330., 116., 4., 5. , 4.5, 3.78 , 1., 0.91),
       (400., 333., 117., 4., 5. , 4. , 3.864, 1., 0.95)],
      dtype=[('Número_de_Série', '<f8'), ('Pontuação_GRE', '<f8'),
('Pontuação_TOEFL', '<f8'), ('Classificação_da_Universidade', '<f8'), ('SOP',
'<f8'), ('LOR', '<f8'), ('CGPA', '<f8'), ('Pesquisa', '<f8'),
('Chance_de_Admissao', '<f8')])
```

Isso mostra 117 alunos com chance de admissão maior que 0.8, porém, vem com todas as informações, porque na realidade estamos pegando todas as linhas que a condição é satisfeita. Quando colocamos o ['Pontuação_GRE'], estamos filtrando, dessas linhas, apenas a coluna 'Pontuação_GRE'.

[155]:
```
# Vamos fazer isso com CGPA também
print(admissao_graduacao[admissao_graduacao['Chance_de_Admissao'] > 0.
↪8]['CGPA'].mean())
print(admissao_graduacao[admissao_graduacao['Chance_de_Admissao'] < 0.
↪4]['CGPA'].mean())
```

```
3.7106666666666666
3.0222857142857142
```

Era de se esperar isso, o GPA e o GRE para alunos com maior chance de serem admitidos, pela análise superficial, parecem ser maiores.

09_regex_matching_ancoras

February 27, 2026

1 Regex Matching e âncoras

Correspondência de padrões em cadeias de caracteres (str) usando expressões regulares. Expressões regulares (ou regexes) são escritas em uma linguagem de formatação condensada.

Em geral, você pode pensar na expressão regular como um padrão que você fornece a um processador regex com alguns dados de origem. Em seguida, o processador analisa esses dados de origem usando o padrão e retorna trechos de texto para cientista de dados ou programador para posterior manipulação. Existem três razões principais pelas quais você deve querer fazer isso. - 1° Para verificar se existe um padrão em alguns dados de origem. - 2° Para obter todas as instâncias de um padrão complexo de alguns dados de origem. - 3° Para limpar seus dados de origem usando um padrão, geralmente por meio da divisão de cadeias.

Os regexes não são triviais, mas são uma técnica fundamental para limpeza de dados em aplicativos de cîencia de dados. Além disso, uma sólida compreensão dos regexes ajudará você a manipular dados de texto de forma rápida e eficiente para outras aplicações de ciência de dados. Vamos entender um pouco sobre o **BÁSICO** de regex.

RESUMINDO: Regex (abreviação de regular expressions) são expressões regulares usadas para procurar, extrair, validar ou substituir padrões em textos (strings).

E o que é uma âncora afinal? R: Uma âncora é um elemento que não corresponde a um caractere, mas sim a uma posição no texto. Ela diz ONDE o padrão deve ocorrer, não o que deve ocorrer.

Padrões simples

[1]:
```
# Primeiro, vamos importar o módulo RE, que é onde o Python armazena␣
↪bibliotecas  de expressões regulares.
import re
```

[ ]:
```
# Há varias funções principais de processamento no RE que você pode usar.

# A primeira correspondência verifica se há uma correspondência de string que␣
↪esteja no início da string e retorna um booleano. Da mesma forma, a pesquisa␣
↪verifica uma correspondência em qualquer lugar da string e retorna um␣
↪booleano.

# EXEMPLO:
texto = 'Esse é um texto de exemplo. Bom dia!'
```

```
# Vamos verificar se esse texto possui bom dia.
if re.search('bom', texto.lower()):
    print('Existe')

else:
    print('Não existe')

# OBS: também dá para fazer isso com "if 'bom' in text.lower():"
```

```
Existe
```

[ ]:
```
# Além de verificar as condicionais, podemos segmentar uma string. O trabalho␣
↪que o regex faz aqui é chamado de tokenização, em que a string é separa em␣
↪substrings com base em padrões.
# A tokenização (tokenizing) é uma atividade central no processamento de␣
↪linguagem natural.

# As funções findall e split analisarão a string para nóse retornarão pedaços.
# EXEMPLO:
texto = 'Amy trabalha diligentemente. Amy tira boas notas. Nossa aluna Amy é␣
↪bem-sucedida.'

# Vamos dividir isso em todas as instâncias de Amy.
re.split('Amy', texto)

# Também dá para fazer da seguinte forma: "texto.split('Amy')"
```

[ ]:
```
['',
 ' trabalha diligentemente. ',
 ' tira boas notas. Nossa aluna ',
 ' é bem-sucedida.']
```

[22]:
```
# Perceba que o split() retornou uma str vazia seguida por várias declarações␣
↪sobre Amy, todas como elementos de uma lista.
# Se quiséssemos contar quantas vezes falamos sobre Amy, poderíamos usar␣
↪findall().

re.findall('Amy', texto)

# Também é possível usa o count() como "texto.count('Amy')". A Diferença é que␣
↪com 'count()' será mostrado um número de vezes ao invés de uma lista com as␣
↪repetidas frases.
```

[22]:
```
['Amy', 'Amy', 'Amy']
```

[ ]:
```
# Procurar números em um texto
texto = "O aluno tirou 750 no GRE e 105 no TOEFL"
```

```
numeros = re.findall(r'\d+', texto)
print(numeros)

# ATENÇÃO: O 'r' antes equivale a dizer para o Python não tentar interpretar o␣
↪\d (isso evita bugs). Por exemplo, se fosse '\n' o Python pula a linha,␣
↪porém se tiver r'\n, ele não pula a linha (se fosse dentro de um print, ele␣
↪imprimiria na tela o '\n').
```

```
['750', '105']
```

Vimos que o .search() procura por algum padrão e retorna um booleano. O split() usará um padrão para criar um lista de substrings. E esse findall() procurará um padrão e retirará todas as ocorrências.

Padrões mais complexos

[26]:
```
# O padrão de especificação regex define uma linguagem de marcação para␣
↪descrever padrões no texto.

# Vamos começar com âncoras, pois elas especificam o início e/ou o fim da␣
↪string que você está tentando combinar. O caractere '^' significa início e o␣
↪caractere '$' significa o fim.
# Se você colocar o cursor antes de uma string, isso significa que o texto que␣
↪o processador regex recupera deve começar com a string que você especificar.
# Para finalizar, você deve colocar o caractere '$' após a string. Isso␣
↪significa que o texto recuperado pelo regex deve terminar com a string que␣
↪você especificar.

# EXEMPLO:
texto = 'Amy trabalha diligentemente. Amy tira boas notas. Nossa aluna Amy é␣
↪bem-sucedida.'

# Vamos ver se isso começa com 'Amy'.
re.search('^Amy',texto)
```

[26]:
```
<re.Match object; span=(0, 3), match='Amy'>
```

Perceba que o search() retornou para nós um novo objeto chamado “Match object”. Esse objeto sempre tem um valor booleano verdadeiro quando algo foi encontrado. Portanto, você sempre pode avaliá-lo em uma declaração “if”, como foi feito anteriormente. A renderização do objeto de correspondência também informa qual padrão foi correspondido. Nesse caso, a palavra ‘Amy’ e o local em que a correspondência estava.

Regex	Significado
.	Qualquer caractere
\d	Dígito (0–9)

Regex	Significado
`\w`	Letra ou número
`+`	Uma ou mais vezes
`*`	Zero ou mais
`?`	Zero ou uma
`^`	Início da string
`$`	Final da string
`[abc]`	Um dos caracteres
`[0-9]`	Intervalo

Símbolos mais comuns

10_classes_caracteres

February 27, 2026

1 Padrões de classes e caracteres (conjunto)

[18]:
```
import re
```

[19]:
```
# Vamos criar uma série de notas de alunos individuas ao longo de um semestre␣
↪em um curso em todas as suas tarefas individuais.

notas = "ACAAAABCBCBAA"

# Se quisermos responder à pergunta, quantos Bs estavam na lista de notas,␣
↪poderíamos simplesmente usar B como o nosso padrão

re.findall('B', notas)
```

[19]:
```
['B', 'B', 'B']
```

[20]:
```
# Se quisermos contar o número de 'A's ou 'B's na lista, não podemos usar A e␣
↪B, pois isso é usado para corresponder a todos os 'A's seguidos␣
↪imediatamente por 'B' (será considerado a str "AB").

# Para fazer do jeito que queremos, passaremos a string dentro de []
re.findall('[AB]', notas)
```

[20]:
```
['A', 'A', 'A', 'A', 'A', 'B', 'B', 'B', 'A', 'A']
```

[ ]:
```
# Você também pode incluir uma variedade de caracteres ordenados␣
↪alfanumericamente. Por exemplo, se quisermos nos referir a todos os␣
↪caracteres minúsculos, podemos usar [a-z] nos colchetes.

# Vamos criar um regex simples para analisar todas as intâncias em que o aluno␣
↪recebe um A seguido por um B ou um C. Então:

re.findall('[A][B-C]', notas)

# re.findall('A[B-C]', notas) também funciona.
```

[ ]:
```
['AC', 'AB']
```

Perceba como o padrão A, B descreve um conjunto de caracteres possíveis que podem ser A ou B, enquanto o padrão A e seguido pelo padrão B-C denotam dois conjuntos de caracteres que devem ter sido combinados consecutivamente.

[22]:
```
# Você pode escrever esse padrão usando o operador pipe "|", que significa OU.

re.findall('AB|AC', notas)
```

[22]:
```
['AC', 'AB']
```

[23]:
```
# Podemos usar o cursor com o operador set (o operador set no contexto de regex␣
↪é "[]"") para negar nossos resultados "^". Por exemplo, se quiséssemos␣
↪analisar apenas as notas que não eram 'A's, faríamos:

re.findall('[^A]', notas)
```

[23]:
```
['C', 'B', 'C', 'B', 'C', 'B']
```

[ ]:
```
# Perceba que o símbolo ^ normalmente indica o início da string. Porém, quando␣
↪ele aparece dentro de um conjunto de caracteres ([]), ele deixa de ser uma␣
↪âncora e passa a ter outro significado (negação), assim como outros␣
↪caracteres especiais também perdem seus significados originais.

re.findall('^[^A]', notas)

# Regex é contextual, isto é, o mesmo símbolo pode ter significados diferentes␣
↪dependendo de onde aparece. Nesse caso, ele deixa de ser âncora.
```

[ ]:
```
[]
```

É uma lista vazia porque o regex está dizendo que queremos corresponder a qualquer valor no início da string, que não seja um A, e nossa string começa com um A.

Quantificadores

Os quantificadores são o número de vezes que você deseja que um padrão seja correspondido para realmente contar como uma correspondência.
O quantificador mais básico é a expressão de e{m,n}, em que 'e' é a expressão ou caractere que estamos combinando, 'm' é o número mínimo de vezes que você deseja que ele seja correspondido, e 'n' é o número máximo de vezes que o item pode ser correspondido.

ATENÇÃO:

Nesse caso, não pode haver espaço entre m e n, isto é e{m,n} é diferente de e{m, n}

In [1]:
```
import re
```

In [8]:
```
notas = "ACAAAABCBCBAA"

# Quantas vezes esse aluno teve uma sequência consecutiva de A?

re.findall('A{2,10}', notas)
# 2 é o nosso mínimo e 10 é o nosso máximo
```

Out[8]:
```
['AAAA', 'AA']
```

Perceba que temos daus sequências, uma em que o aluno tirou quatro 'A's seguidos e uma em que ele tirou dois 'A's seguidos.

In [ ]:
```
# Podemos tentar fazer isso usando valores únicos e apenas
# repetindo o padrão.

re.findall('A{1,1}A{1,1}', notas)
```

Out[ ]:
```
['AA', 'AA', 'AA']
```

Como pode ver, isso é diferente do primeiro exemplo. O primeiro padrão procura uma combinação de dois 'A's até dez 'A's em uma linha. Dessa forma, ele vê quatro 'A's como uma sequência.
O segundo padrão é procurar dois 'A's consecutivos. Ele vê dois 'A's seguidos imediatamente por mais dois 'A's.

Dizemos que o processador regex começa no início da string e consome variáveis que correspondem aos padrões da mesma forma que consome.

In [ ]:
```
# É importante notar que a sintaxe do quantificador regex não
# permite que você se desvie do padrão {m,n}, ou seja, se você
#  tiver um espaço entre as chaves obterá um resultado vazio.

re.findall('A{2, 10}', notas)
```

Out[ ]:
```
[]
```

In [ ]:
```
# Como já vimos, se não incluirmos um quantificador quando o padrão
# for {1,1}, podemos colocar os caracteres de forma consecutiva.

re.findall('AA', notas)
```

Out[ ]:
```
['AA', 'AA', 'AA']
```

In [ ]:
```
# Se você tem um número entre chaves, ele é considerado o valor m
#  e o valor n, então:

re.findall('A{2}', notas)

# indica que o 2 é o valor mínimo e o valor máximo.
```

Out[ ]:
```
['AA', 'AA', 'AA']
```

In [ ]:
```
# Usando isso, podemos encontrar uma tendência decrescente nas notas
#  de um aluno, por exemplo:

re.findall('A{1,10}B{1,10}C{1,10}', notas)
```

Out[ ]:
```
['AAAABC']
```

Isso é um pouco complicado, porque incluímos um máximo, que era arbitrariamente grande (parece gambiarra).

Existem outros três quantificadores usados como abreviatura que poderíamos pensar em usar aqui.

- Um asterisco '*', que é usado para combinar zero ou mais vezes.
- Um ponto de interrogação '?' para coincidir uma ou mais vezes.
- Um sinal de mais '+' para coincidir uma ou mais vezes.

In [18]:
```
# Exemplo mais complexo:
with open('datasets/ferpa.txt', 'r') as dados:
    # Vamos ler isso em uma variável chamada wiki
    wiki = dados.read()


print(wiki)
```

Overview[edit]
FERPA gives parents access to their child's education records, an opportunity to seek to have the records amended, and some control over the disclosure of information from the records. With several exceptions, schools must have a student's consent prior to the disclosure of education records after that student is 18 years old. The law applies only to educational agencies and institutions that receive funds under a program administered by the U.S. Department of Education.

Other regulations under this act, effective starting January 3, 2012, allow for greater disclosures of personal and directory student identifying information and regulate student IDs and e-mail addresses.[2] For example, schools may provide external companies with a student's personally identifiable information without the student's consent.[2]

Examples of situations affected by FERPA include school employees divulging information to anyone other than the student about the student's grades or behavior, and school work posted on a bulletin board with a grade. Generally, schools must have written permission from the parent or eligible student in order to release any information from a student's education record.

This privacy policy also governs how state agencies transmit testing data to federal agencies, such as the Education Data Exchange Network.

This U.S. federal law also gave students 18 years of age or older, or students of any age if enrolled in any post-secondary educational institution, the right of privacy regarding grades, enrollment, and even billing information unless the school has specific permission from the student to share that specific type of information.

FERPA also permits a school to disclose personally identifiable information from education records of an "eligible student" (a student age 18 or older or enrolled in a postsecondary institution at any age) to his or her parents if the student is a "dependent student" as that term is defined in Section 152 of the Internal Revenue Code. Generally, if either parent has claimed the student as a dependent on the parent's most recent income tax statement, the school may non-consensually disclose the student's education records to both parents.[3]

The law allowed students who apply to an educational institution such as graduate school permission to view recommendations submitted by others as part of the application. However, on standard application forms, students are given the option to waive this right.

FERPA specifically excludes employees of an educational institution if they are not students.

The act is also referred to as the Buckley Amendment, for one of its proponents, Senator James L. Buckley of New York.

Access to public records[edit]
The citing of FERPA to conceal public records that are not "educational" in nature has been widely criticized, including by the act's primary Senate sponsor.[4] For example, in the Owasso Independent School District v. Falvo case, an important part of the debate was determining the relationship between peer-grading and "education records" as defined in FERPA. In the Court of Appeals, it was ruled that students placing grades on the work of other students made such work into an "education record." Thus, peer-grading was determined as a violation of FERPA privacy policies because students

```
had access to other students' academic performance without full consent.
[5] However, when the case went to the Supreme Court, it was officially ru
led that peer-grading was not a violation of FERPA. This is because a grad
e written on a student's work does not become an "education record" until
the teacher writes the final grade into a grade book.[6]

Student medical records[edit]
Legal experts have debated the issue of whether student medical records (f
or example records of therapy sessions with a therapist at an on-campus co
unseling center) might be released to the school administration under cert
ain triggering events, such as when a student sued his college or universi
ty.[7][8]

Usually, student medical treatment records will remain under the protectio
n of FERPA, not the Health Insurance Portability and Accountability Act (H
IPAA). This is due to the "FERPA Exception" written within HIPAA.[9]
```

In [ ]:
```
# Ao examinar esse documento, uma das coisas que notamos é que todos
# os cabeçalhos têm a palavra 'edit' entre colchetes
# Se quiséssemos obter uma lista de todos os cabeçalhos deste artigo,
#  poderíamos fazer isso usando re.findal()

re.findall('[a-zA-Z]{1,100}\[edit\]', wiki)
# Estamos procurando uma palavra que possui de a-zA-Z que tenha de 1
# até 100 dessas letras seguida por [edit].
# MAS POR QUE DAS '\' ?
# R: Lembrando que existem caracteres especiais, como '\n', então se
# coloca uma '\' antes do caractere especial, que nesse caso são os '[]'
# para indicar ao interpretador do Python que aquilo não é para ser um
# caractere especial.
```

Out[ ]:
```
['Overview[edit]', 'records[edit]', 'records[edit]']
```

In [ ]:
```
# Isso não funcionou muito bem. Tem como deixar mais compacto.

# Vamos melhorar isso de forma iterativa.
# Primeiro, podemos usar \w para combinar qualquer letra, incluindo
# dígitos e números.

re.findall('[\w]{1,100}\[edit\]', wiki)
# O '\w' substituiu qualquer letra de a-zA-Z e números.
```

Out[ ]:
```
['Overview[edit]', 'records[edit]', 'records[edit]']
```

O '\w' é um metacaractere e indica um padrão especial de qualquer letra ou dígito. Na verdade, existem vários metacaractere (olhe a tabela da aula 9, lá tem alguns dos mais usados)

ATENÇÃO:

- Pesquise sobre a documentação dos metacaracteres

In [ ]:
```
# Há outros três quantificadores que podemos usar para encurtar
# a sintaxe do curly brace. Podemos usar o '*' para combinar zero
# ou mais vezes.

re.findall('[\w]*\[edit\]', wiki)
```

Out[ ]: ['Overview[edit]', 'records[edit]', 'records[edit]']

In [ ]:
```
# Agora que reduzimos o regex, vamos melhorá-lo um pouco mais.
# Podemos adicionar espaços usando o caractere de espaço.

re.findall('[\w ]*\[edit\]', wiki)
```

Out[ ]:
```
['Overview[edit]',
 'Access to public records[edit]',
 'Student medical records[edit]']
```

In [ ]:
```
# Isso nos gerou uma lista de títulos de seções na página da
# Wikipedia. Agora podemos criar uma lista de títulos fazendo uma
# iteração e aplicando outra regex

for titulo in re.findall('[\w ]*\[edit\]', wiki):
    # Vamos pegar o resultado intermediário, dividir sempre que
    # aparecer um colchetes e pegar o primeiro resultado
    #print(re.split('[\[]', titulo)[0])
    titulo = str(titulo).split('[')
    print(titulo[0])
```

```
Overview
Access to public records
Student medical records
```

Dessa forma, pegamos apenas os títulos reais

12_grupos

February 27, 2026

1 Grupos

Até agora falamos de regex como um único padrão, no qual é correspondido, mas você pode realmente combinar padrões diferentes, chamados de grupos ao mesmo tempo, e depois se referir a esses grupos mais tarde, conforme desejar. Para agrupar padrões, basta usar parênteses. Vamos fazer o que fizemos em Quantificadores usando grupos agora.

[3]:
```
import re
```

[4]:
```
# Lendo o arquivo de texto da Wiki
with open('datasets/ferpa.txt', 'r') as arquivo:
    wiki = arquivo.read()

print(wiki)
```

```
Overview[edit]
FERPA gives parents access to their child's education records, an opportunity to
seek to have the records amended, and some control over the disclosure of
information from the records. With several exceptions, schools must have a
student's consent prior to the disclosure of education records after that
student is 18 years old. The law applies only to educational agencies and
institutions that receive funds under a program administered by the U.S.
Department of Education.

Other regulations under this act, effective starting January 3, 2012, allow for
greater disclosures of personal and directory student identifying information
and regulate student IDs and e-mail addresses.[2] For example, schools may
provide external companies with a student's personally identifiable information
without the student's consent.[2]

Examples of situations affected by FERPA include school employees divulging
information to anyone other than the student about the student's grades or
behavior, and school work posted on a bulletin board with a grade. Generally,
schools must have written permission from the parent or eligible student in
order to release any information from a student's education record.

This privacy policy also governs how state agencies transmit testing data to
federal agencies, such as the Education Data Exchange Network.
```

This U.S. federal law also gave students 18 years of age or older, or students of any age if enrolled in any post-secondary educational institution, the right of privacy regarding grades, enrollment, and even billing information unless the school has specific permission from the student to share that specific type of information.

FERPA also permits a school to disclose personally identifiable information from education records of an "eligible student" (a student age 18 or older or enrolled in a postsecondary institution at any age) to his or her parents if the student is a "dependent student" as that term is defined in Section 152 of the Internal Revenue Code. Generally, if either parent has claimed the student as a dependent on the parent's most recent income tax statement, the school may non-consensually disclose the student's education records to both parents.[3]

The law allowed students who apply to an educational institution such as graduate school permission to view recommendations submitted by others as part of the application. However, on standard application forms, students are given the option to waive this right.

FERPA specifically excludes employees of an educational institution if they are not students.

The act is also referred to as the Buckley Amendment, for one of its proponents, Senator James L. Buckley of New York.

Access to public records[edit]

The citing of FERPA to conceal public records that are not "educational" in nature has been widely criticized, including by the act's primary Senate sponsor.[4] For example, in the Owasso Independent School District v. Falvo case, an important part of the debate was determining the relationship between peer-grading and "education records" as defined in FERPA. In the Court of Appeals, it was ruled that students placing grades on the work of other students made such work into an "education record." Thus, peer-grading was determined as a violation of FERPA privacy policies because students had access to other students' academic performance without full consent.[5] However, when the case went to the Supreme Court, it was officially ruled that peer-grading was not a violation of FERPA. This is because a grade written on a student's work does not become an "education record" until the teacher writes the final grade into a grade book.[6]

Student medical records[edit]

Legal experts have debated the issue of whether student medical records (for example records of therapy sessions with a therapist at an on-campus counseling center) might be released to the school administration under certain triggering events, such as when a student sued his college or university.[7][8]

Usually, student medical treatment records will remain under the protection of

FERPA, not the Health Insurance Portability and Accountability Act (HIPAA). This is due to the "FERPA Exception" written within HIPAA.[9]

[5]:
```
# Vamos utilizar grupos
re.findall('([\w ]*)(\[edit\])', wiki)

# O que fizemos aqui foi: Queremos um conjunto de palavras que tenham espaço e␣
↪seguida por [edit]. Quando colocamos os '()', estamos falando para que esse␣
↪conjunto seja um grupo e o [edit] seja outro grupo.
```

[5]:
```
[('Overview', '[edit]'),
 ('Access to public records', '[edit]'),
 ('Student medical records', '[edit]')]
```

[6]:
```
# O módulo RE divide o resultado grupo por grupo. Além disso, também podemos␣
↪nos referir aos grupos por número com os objetos de correspondência.
# Mas o que fazemos se quisermos uma lista de objetos correspondente? Vamos␣
↪utilizar a função findinter(), pois ao invés de uma lista normal, temos uma␣
↪lista de objetos.
for item in re.finditer('([\w ]*)(\[edit\])', wiki):
    print(item.groups())
```

```
('Overview', '[edit]')
('Access to public records', '[edit]')
('Student medical records', '[edit]')
```

[7]:
```
# Podemos obter um grupo individual usando o sub número do grupo em que o grupo␣
↪abaixo de zero é a parte inteira e o outro número é a parte da parte em que␣
↪estávamos interessados.
# Nesse caso, queremos o grupo sub um, então não queremos a parte inteira␣
↪(seria 'group(1)').

for item in re.finditer('([\w ]*)(\[edit\])', wiki):
    print(item.group()) # Grupo inteiro

print()

for item in re.finditer('([\w ]*)(\[edit\])', wiki):
    print(item.group(1)) # Apenas a primeira parte do grupo
```

```
Overview[edit]
Access to public records[edit]
Student medical records[edit]

Overview
Access to public records
Student medical records
```

[8]:
```
# Uma outra forma seria nomear ou rotular os grupos. No exemplo anterior, foi␣
↪mostrado como pode usar a posição do grupo, porém dar a eles um rótulo e ver␣
↪os resultados como um dicionário é mais útil.
# Para isso, vamos usar a sintaxe (?P<name>).
# O "?P" indicia que essa é uma extensão dos regexes básicos. E o nome nos␣
↪colchetes angulares (<nome>) é a chave do dicionário que usamos.

for item in re.finditer('(?P<titulo>[\w ]*)(?P<edit>\[edit\])', wiki):
    # Podemos obter o retorno do dicionário para o item usando o .groupdict()
    print(item.groupdict()['titulo'])

# Também funciona com o group()
print()

for item in re.finditer('(?P<titulo>[\w ]*)(?P<edit>\[edit\])', wiki):
    print(item.group('titulo'))
```

```
Overview
Access to public records
Student medical records

Overview
Access to public records
Student medical records
```

[9]:
```
# Também poderíamos imprimir o dicionário inteiro
print(item.groupdict())
```

```
{'titulo': 'Student medical records', 'edit': '[edit]'}
```

13_assercoes_app_avancados

February 27, 2026

1 Asserções e aplicativos avançados

1.0.1 Look-ahead e Look-behind

Nesse caso, o padrão que está sendo combinado com o mecanismo de regex é para o texto antes ou depois do texto que estamos realmente tentando isolar. Por exemplo, em nosso cabeçalhos, queremos isolar o texto que vem antes da renderização de [edit], mas na verdade não nos importamos com o texto [edit] em si. Até agora estamos descartando o [edit], mas se quisermos usá-los para combinar, mas não quisermos capturá-los, podemos colocá-los em um grupo e usar lookahead em vez disso através de ?= na sintaxe.

[1]:
```
import re
```

[2]:
```
# Lendo o arquivo de texto da Wiki
with open('datasets/ferpa.txt', 'r') as arquivo:
    wiki = arquivo.read()

print(wiki)
```

```
Overview[edit]
FERPA gives parents access to their child's education records, an opportunity to
seek to have the records amended, and some control over the disclosure of
information from the records. With several exceptions, schools must have a
student's consent prior to the disclosure of education records after that
student is 18 years old. The law applies only to educational agencies and
institutions that receive funds under a program administered by the U.S.
Department of Education.

Other regulations under this act, effective starting January 3, 2012, allow for
greater disclosures of personal and directory student identifying information
and regulate student IDs and e-mail addresses.[2] For example, schools may
provide external companies with a student's personally identifiable information
without the student's consent.[2]

Examples of situations affected by FERPA include school employees divulging
information to anyone other than the student about the student's grades or
behavior, and school work posted on a bulletin board with a grade. Generally,
schools must have written permission from the parent or eligible student in
order to release any information from a student's education record.
```

This privacy policy also governs how state agencies transmit testing data to federal agencies, such as the Education Data Exchange Network.

This U.S. federal law also gave students 18 years of age or older, or students of any age if enrolled in any post-secondary educational institution, the right of privacy regarding grades, enrollment, and even billing information unless the school has specific permission from the student to share that specific type of information.

FERPA also permits a school to disclose personally identifiable information from education records of an "eligible student" (a student age 18 or older or enrolled in a postsecondary institution at any age) to his or her parents if the student is a "dependent student" as that term is defined in Section 152 of the Internal Revenue Code. Generally, if either parent has claimed the student as a dependent on the parent's most recent income tax statement, the school may non-consensually disclose the student's education records to both parents.[3]

The law allowed students who apply to an educational institution such as graduate school permission to view recommendations submitted by others as part of the application. However, on standard application forms, students are given the option to waive this right.

FERPA specifically excludes employees of an educational institution if they are not students.

The act is also referred to as the Buckley Amendment, for one of its proponents, Senator James L. Buckley of New York.

Access to public records[edit]

The citing of FERPA to conceal public records that are not "educational" in nature has been widely criticized, including by the act's primary Senate sponsor.[4] For example, in the Owasso Independent School District v. Falvo case, an important part of the debate was determining the relationship between peer-grading and "education records" as defined in FERPA. In the Court of Appeals, it was ruled that students placing grades on the work of other students made such work into an "education record." Thus, peer-grading was determined as a violation of FERPA privacy policies because students had access to other students' academic performance without full consent.[5] However, when the case went to the Supreme Court, it was officially ruled that peer-grading was not a violation of FERPA. This is because a grade written on a student's work does not become an "education record" until the teacher writes the final grade into a grade book.[6]

Student medical records[edit]

Legal experts have debated the issue of whether student medical records (for example records of therapy sessions with a therapist at an on-campus counseling center) might be released to the school administration under certain triggering

events, such as when a student sued his college or university.[7][8]

Usually, student medical treatment records will remain under the protection of FERPA, not the Health Insurance Portability and Accountability Act (HIPAA). This is due to the "FERPA Exception" written within HIPAA.[9]

[5]:
```
# Vamos usar o lookahead

for item in re.finditer('(?P<titulo>[\w ]+)(?=\[edit\])', wiki):
    # O segundo grupo é, na verdade, um desses grupos descartáveis que são␣
 ↪lookahead, então estamos procurando um [edit], mas não estamos colocando ele␣
 ↪no objeto.
    print(item)
```

```
<re.Match object; span=(0, 8), match='Overview'>
<re.Match object; span=(2715, 2739), match='Access to public records'>
<re.Match object; span=(3692, 3715), match='Student medical records'>
```

Vamos ver mais uns dados:

[15]:
```
with open('datasets/buddhist.txt', 'r') as arquivo:
    wiki = arquivo.read()

print(wiki)
```

```
Buddhist universities and colleges in the United States
From Wikipedia, the free encyclopedia
Jump to navigationJump to search

This article needs additional citations for verification. Please help improve
this article by adding citations to reliable sources. Unsourced material may be
challenged and removed.
Find sources: "Buddhist universities and colleges in the United States" - news ·
newspapers · books · scholar · JSTOR (December 2009) (Learn how and when to
remove this template message)
There are several Buddhist universities in the United States. Some of these have
existed for decades and are accredited. Others are relatively new and are either
in the process of being accredited or else have no formal accreditation. The
list includes:

Dhammakaya Open University - located in Azusa, California, part of the Thai Wat
Phra Dhammakaya[1]
Dharmakirti College - located in Tucson, Arizona Now called Awam Tibetan
Buddhist Institute (http://awaminstitute.org/)
Dharma Realm Buddhist University - located in Ukiah, California
Ewam Buddhist Institute - located in Arlee, Montana
Naropa University is located in Boulder, Colorado (Accredited by the Higher
Learning Commission)
Institute of Buddhist Studies - located in Berkeley, California
```

```
Maitripa College - located in Portland, Oregon
Soka University of America - located in Aliso Viejo, California
University of the West - located in Rosemead, California
Won Institute of Graduate Studies - located in Glenside, Pennsylvania
References[edit]
^ Banchanon, Phongphiphat (3 February 2015).    "        " [Getting
to know the Dhammakaya network]. Forbes Thailand (in Thai). Retrieved 11
November 2016.
External links[edit]
List of Buddhist Universities and Colleges in the world
```

[31]:
```
# Podemos ver que cada universidade segue um padrão bastante semelhante, com o␣
↪nome seguido por um traço '-' e com a localização da universidade.

# Vamos utilizar o modo mais detalhado do regex, o qual permite que você␣
↪escreva expressões regulares de várias linhas e aumenta a legibilidade

padrao = """
(?P<titulo>.*)                  # Título da universidade
(-\ located\ in\ )          # Indicador da localização
(?P<cidade>\w*)                 # Cidade da universidade
(,\ )                           # Separador de estado
(?P<estado>\w*)                 # Estado da universidade
"""

# Agora, quando chamamos o findinter, apenas passamos o sinalizador re.VERBOSE␣
↪como último parâmetro. Isso torna muito mais fácil entender grandes regexes.
for item in re.finditer(padrao, wiki, re.VERBOSE):
    # Com o re.VERBOSE conseguimos escrever a regex de forma legível, ignorando␣
↪espaços em branco e comentários (#) dentro do padrão.
    print(item.groupdict())
```

```
{'titulo': 'Dhammakaya Open University ', 'cidade': 'Azusa', 'estado':
'California'}
{'titulo': 'Dharmakirti College ', 'cidade': 'Tucson', 'estado': 'Arizona'}
{'titulo': 'Dharma Realm Buddhist University ', 'cidade': 'Ukiah', 'estado':
'California'}
{'titulo': 'Ewam Buddhist Institute ', 'cidade': 'Arlee', 'estado': 'Montana'}
{'titulo': 'Institute of Buddhist Studies ', 'cidade': 'Berkeley', 'estado':
'California'}
{'titulo': 'Maitripa College ', 'cidade': 'Portland', 'estado': 'Oregon'}
{'titulo': 'University of the West ', 'cidade': 'Rosemead', 'estado':
'California'}
{'titulo': 'Won Institute of Graduate Studies ', 'cidade': 'Glenside', 'estado':
'Pennsylvania'}
```

Outro exemplo, agora do New York Times:

[32]:
```
# Lendo o arquivo
with open('datasets/nytimeshealth.txt', 'r') as arquivo:
    noticia = arquivo.read()

print(noticia)
```

```
548662191340421120|Sat Dec 27 02:10:34 +0000 2014|Risks in Using Social Media to
Spot Signs of Mental Distress http://nyti.ms/1rqi9I1
548579831169163265|Fri Dec 26 20:43:18 +0000 2014|RT @paula_span: The most
effective nationwide diabetes prevention program you've probably never heard of:
http://newoldage.blogs.nytimes.com/2014/12/26/diabetes-prevention-that-works/
548579045269852161|Fri Dec 26 20:40:11 +0000 2014|The New Old Age Blog: Diabetes
Prevention That Works http://nyti.ms/1xm7fTi
548444679529041920|Fri Dec 26 11:46:15 +0000 2014|Well: Comfort Casseroles for
Winter Dinners http://nyti.ms/1xTNoO0
548311901227474944|Fri Dec 26 02:58:39 +0000 2014|High-Level Knowledge Before
Veterans Affairs Scandal http://nyti.ms/13yCpvS
548305625449787392|Fri Dec 26 02:33:42 +0000 2014|Your Money: Affordable Care
Act's Tax Effects Now Loom for Filers http://nyti.ms/13yAtUf
548283182853160960|Fri Dec 26 01:04:32 +0000 2014|Well: Christmas in the
Hospital http://nyti.ms/1vtPNcm
548278414504108033|Fri Dec 26 00:45:35 +0000 2014|The Texas Tribune: For Members
of Health Ministries in Texas, Caring Means Sharing the Bills
http://nyti.ms/1wLtAen
548176399299649536|Thu Dec 25 18:00:12 +0000 2014|A q&amp;amp;a with Laura
Hillenbrand, author of "Unbroken," who suffers from chronic fatigue syndrome.
http://bit.ly/1ziCflj
547949409288609792|Thu Dec 25 02:58:14 +0000 2014|Race to Deliver Nicotine's
Punch, With Less Risk  http://nyti.ms/1xNzyg4
547904193961656321|Wed Dec 24 23:58:34 +0000 2014|The New Health Care: People
Are Shopping for Health Insurance, Surprisingly http://nyti.ms/1vpIjqE
547904193001185280|Wed Dec 24 23:58:33 +0000 2014|Machine Learning: Bedtime
Technology for a Better Night's Sleep http://nyti.ms/16RCgpE
547872136304394242|Wed Dec 24 21:51:10 +0000 2014|Breaking: Lab Error May Have
Exposed U.S. Technician to Ebola Virus http://nyti.ms/1ziIYf6
547870756592300034|Wed Dec 24 21:45:41 +0000 2014|C.D.C. Ebola Error in Lab May
Have Exposed Technician to Virus http://nyti.ms/1CDkLqc
547870381160140800|Wed Dec 24 21:44:12 +0000 2014|RT @nytimes: Breaking News:
Lab Error May Have Exposed U.S. Technician to Ebola Virus http://nyti.ms/1B2ThGD
547802935141232642|Wed Dec 24 17:16:12 +0000 2014|A q&amp;amp;a with Laura
Hillenbrand, author of "Unbroken," who suffers from chronic fatigue syndrome.
http://nyti.ms/1B5PO9Q
547765704829964289|Wed Dec 24 14:48:15 +0000 2014|RT @cslnyt: Virus That Killed
a Kansas Farmer is identified @CDCgov  @nytDeniseGrady
http://www.nytimes.com/2014/12/24/science/mysterious-virus-that-killed-a-farmer-
in-kansas-is-identified.html?smid=nytcore-iphone-share&smprod=nytcore-iphone
```

```
Reduction in Tonsillectomies Teaches Us About Medicine http://nyti.ms/1m2xmd9
483624988629811200|Mon Jun 30 14:55:57 +0000 2014|RT @WilliamJBroad: Why
Teenagers Act Crazy http://nyti.ms/1qmj9Ls
483624693501816832|Mon Jun 30 14:54:46 +0000 2014|RT @jswatz: Facebook tinkering
with our feeds to emotional contagion. Should I feel Hawthorne effect or
Stockholm Syndrome? http://t.co/xOp…
483624639047172096|Mon Jun 30 14:54:33 +0000 2014|RT @cslnyt: Devastating errors
after military care, a NYT investigation shows  http://nyti.ms/1wVzVBP
483624560743706624|Mon Jun 30 14:54:15 +0000 2014|RT @David_Dobbs: Pistorius Was
Not Mentally Ill at Time of Shooting, Psychiatrists Report, via @nytimes
http://nyti.ms/1m1hWpq
483465680792989697|Mon Jun 30 04:22:55 +0000 2014|Well: Keeping Your Cool
http://nyti.ms/1ogUkxo
483461727108235264|Mon Jun 30 04:07:12 +0000 2014|World Cup 2014: With Júlio
César's Help, More Respect for Brazilians in Net http://nyti.ms/1ogST1W
483457847091789824|Mon Jun 30 03:51:47 +0000 2014|Facebook Tinkers With Users'
Emotions in News Feed Experiment, Stirring Outcry http://nyti.ms/1z0Z2VJ
483453606751125504|Mon Jun 30 03:34:56 +0000 2014|Medical Boards Draft Plan to
Ease Path to Out-of-State and Online Treatment http://nyti.ms/1ogQy7e
483453605899673600|Mon Jun 30 03:34:56 +0000 2014|Flawed Saudi Response Fueled
Outbreak of MERS, Middle East Virus http://nyti.ms/1z0XRFU
483061694596677632|Sun Jun 29 01:37:37 +0000 2014|Cuomo Plan Seeks to End New
York's AIDS Epidemic http://nyti.ms/1msG0O4
482887982362669056|Sat Jun 28 14:07:21 +0000 2014|Technophoria: When a Health
Plan Knows How You Shop http://nyti.ms/1wV0AhZ
482735346208501760|Sat Jun 28 04:00:49 +0000 2014|On Religion: For Women Who Saw
Combat, a Place to Find Inner Peace http://nyti.ms/1lyUccW
482726408004067328|Sat Jun 28 03:25:18 +0000 2014|Afrezza, a New Inhaled
Insulin, Is Approved by the F.D.A. http://nyti.ms/1lyNLqf
482726406934511617|Sat Jun 28 03:25:18 +0000 2014|Report Finds Health Unit of
V.A. Needs Overhaul http://nyti.ms/1lyNLq9
```

[ ]:
```
# Aqui podemos ver que há tweets e os campos são separados por pipes '|'
# Vamos tentar obter uma lista de todas as hashtags incluídas nesses dados.␣
↪Agora, no Twitter, uma hashtag começa com um sinal de libra ou a marca de␣
↪hash e continua até que algumespaço em branco seja encontrado

# Vamos criar um padrão
padrao = '#[\w\d]*(?=\s)'       # OBS: Esse \s inclui qualquer espaço, seja␣
↪ESPAÇO ou TAB

# Vamos ver o que encontraremos
re.findall(padrao, noticia)
```

[ ]:
```
['#askwell',
 '#pregnancy',
 '#Colorado',
```

```
'#VegetarianThanksgiving',
'#FallPrevention',
'#Ebola',
'#Ebola',
'#ebola',
'#Ebola',
'#Ebola',
'#EbolaHysteria',
'#AskNYT',
'#Ebola',
'#Ebola',
'#Liberia',
'#Excalibur',
'#ebola',
'#Ebola',
'#dallas',
'#nobelprize2014',
'#ebola',
'#ebola',
'#monrovia',
'#ebola',
'#nobelprize2014',
'#ebola',
'#nobelprize2014',
'#Medicine',
'#Ebola',
'#Monrovia',
'#Ebola',
'#smell',
'#Ebola',
'#Ebola',
'#Ebola',
'#Monrovia',
'#Ebola',
'#ebola',
'#monrovia',
'#liberia',
'#benzos',
'#ClimateChange',
'#Whole',
'#Wheat',
'#Focaccia',
'#Tomatoes',
'#Olives',
'#Recipes',
'#Health',
'#Ebola',
```

```
 '#Monrovia',
 '#Liberia',
 '#Ebola',
 '#Ebola',
 '#Liberia',
 '#Ebola',
 '#blood',
 '#Ebola',
 '#organtrafficking',
 '#EbolaOutbreak',
 '#SierraLeone',
 '#Freetown',
 '#SierraLeone',
 '#ebolaoutbreak',
 '#kenema',
 '#ebola',
 '#Ebola',
 '#ebola',
 '#ebola',
 '#Ebola',
 '#ASMR',
 '#AIDS2014',
 '#AIDS',
 '#MH17',
 '#benzos']
```

Podemos ver que há muitos tweets relacionados ao Ebola neste conjunto de dados.

2 PARA OLHAR A DOCUMENTAÇÃO DE OPERAÇÕES DE EXPRESSÕES REGULARES, ACESSAR:

https://docs.python.org/3/library/re.html

14_introducao_pandas

February 27, 2026

1 Introdução ao Pandas

Para consultar e compartilhar soluções técnicas reais, utilize: https://stackoverflow.com/questions. Podemos tirar dúvidas, principalmente sobre Pandas nesse site que algum desenvolvedor responderá.

Também é possível utilizar livros: - Python for Data Analysis: https://cdn.bookey.app/files/pdf/book/pt/python-para-an%C3%A1lise-de-dados.pdf - Introdução à Análise de Dados em Saúde com Python: https://docs.bvsalud.org/biblioref/2023/06/1437637/introducao-a-analise-de-dados-em-saude-com-python-ciia-saude.pdf - Learning the Pandas Library

2 A estrutura de dados básica

A série é uma das principais estruturas de dados do Pandas. Podemos pensar na 'série' como um cruzamento entre lista e dicionário. Os itens são todos armazenados em um pedido e há etiquetas com as quais você pode recuperá-los.

[6]:
```
# Importando a biblioteca Pandas
import pandas as pd
```

[10]:
```
# Podemos criar uma série passando uma lista de valores. Quando fazemos isso, o␣
↪Pandas atribui automaticamente um índice começando com zero e define o nome␣
↪da série como None.

# Vamos fazer uma lista com três alunos

estudantes = ['Alice,', 'Jack', 'Molly']

# Agora, chamamos a função de série do Pandas e passamos para os alunos.
pd.Series(estudantes)
```

[10]:
```
0    Alice,
1      Jack
2     Molly
dtype: str
```

Vemos aqui que o Pandas identifica automaticamente o tipo de dados na série. Os valores são indexados com números inteiros, começando com zero.

[11]:
```
# Veremos que se passarmos uma lista de números inteiros, o Pandas define otipo␣
↪como int64. Por trás, o Pandas armazena os valores da série em uma matriz␣
↪digitada usando a biblioteca NumPy, o que oferece uma velocidade␣
↪significativa ao processar dados em comparação com as listas tradicionais do␣
↪Python.

numeros = [1, 2, 3]

# Transformando em uma série.
pd.Series(numeros)
```

[11]:
```
0    1
1    2
2    3
dtype: int64
```

[12]:
```
# Existem alguns detalhes de digitação para o desempenho que é importante␣
↪conhecer. O mais importante é como o NumPy e Pandas lidam com os dados␣
↪perdidos.

# Em Python, temos o tipo None para indicar falta de dados. Porém, None não um␣
↪valor numérico nem um valor "vetorizável", na verdade None é um objeto␣
↪Python especial.
# Tanto o NumPy quanto Pandas trabalham com arrays vetorizados (usufruem de␣
↪operações vetorizadas), os quais precisam de um tipo único (dtype) e de␣
↪operações rápidas em blocos. Dessa forma, nasce um conflito. Como Pandas␣
↪resolve isso?
# Por baixo, o Pandas faz uma conversão de tipo para nós. Se criarmos uma lista␣
↪de str e tivermos um elemento faltante, um tipo None, o Pandas analisa isso␣
↪como uma coluna de determinado tipo, mas com dados faltantes, e, nesse caso,␣
↪ele troca o tipo None para --> NaN (marcados padrão de dado ausente no␣
↪Pandas) para não quebrar operações vetorizadas.

# Vemos criar uma nova lista de estudantes, mas vamos definí-la como None.
estudantes2 = ['Alice', 'Jack', None]
pd.Series(estudantes2)
```

[12]:
```
0    Alice
1     Jack
2      NaN
dtype: str
```

[16]:
```
# Vamos criar uma lista de inteiros e colocamos o tipo None, o Pandas␣
↪automaticamente a converterá em um valor especial de ponto flutuante␣
↪designado como NaN.
numeros2 = [1, 2, None]
pd.Series(numeros2)
```

[16]:
```
0    1.0
1    2.0
2    NaN
dtype: float64
```

Perceba que mesmo tendo inteiros, o Pandas generalizou para Float e utilizou o NaN. Por que não deixar isso como um número inteiro? R: Justamente porque o Pandas considera o NaN, nesse caso, como um tipo float, e assim é utilizado o tipo mais geral, e dessa forma, converteu todos os valores para floats.

OBS: Se tivermos uma coluna com apenas Inteiros (int) e ao transformar em série temos o tipo float, MUITO PROVAVELMENTE estará com dados faltantes.

[17]:
```
# É importante enfatizar que None e NaN podem ser utilizado da mesma forma para␣
↪indicar dados ausentes. CONTUDO, PORÉM, TODAVIA, ENTRETANTO, por baixo dos␣
↪panos, eles NÃO são representados pelo Pandas da mesma forma.

# NaN não é equivalente a None.
import numpy as np      # Biblioteca que permite criarmos o tipo NaN
np.nan == None
```

[17]:
```
False
```

[18]:
```
# Também não podemos fazer o teste de qualidade do NaN sozinho
np.nan == np.nan
```

[18]:
```
False
```

[27]:
```
# Para verificar o tipo, temos que usar funções especiais, como utilizamos a␣
↪isinstance para verificar os próprios tipos do Python.

np.isnan(np.nan)
```

[27]:
```
np.True_
```

Lembre-se que quando vemos um NaN, o seu significado é semelhante a None, mas NaN é um valor numérico e é tratado de forma diferente por motivos de EFICIÊNCIA.

OBS: NaN é um valor especial que o NumPy criou.

15_serie_pandas

February 27, 2026

1 Criando a série Pandas

Como já dito, a Série é uma estrutura de dados do Pandas. Embora a lista possa ser uma forma comum de criar alguns dados, geralmente temos dados com rótulos que desejamos manipular. Uma série pode ser criada diretamente dos dados do dicionário e, se fizermos isso, o índice será automaticamente atribuído às chaves do dicionário.

[1]:
```
# Importando a bilbioteca
import pandas as pd
```

[8]:
```
# Vamos criar um dicionário chamado notas dos estudantes.
aulas_estudantes = {'Alice': 'Física',
                    'Jack': 'Química',
                    'Molly': 'Inglês'}

s = pd.Series(aulas_estudantes)
s
```

[8]:
```
Alice     Física
Jack     Química
Molly     Inglês
dtype: str
```

[4]:
```
# A partir da Série ser criada, podemos obter o objeto de índice usando o␣
↪atributo index.
s.index
```

[4]:
```
Index(['Alice', 'Jack', 'Molly'], dtype='str')
```

[5]:
```
# À medida que utilizamos mais o Pandas, notaremos que muitas coisas são␣
↪implementadas como matrizes numpy e que elas têm o valor dtype definido.␣
↪Isso vale para os índices, e aqui o Pandas inferiu que estamos usando␣
↪strings para o índice.

# Isso é interessante porque o tipo do objeto não é apenas para strings ou␣
↪floats, é para objetos arbitrários (mais complexos).

# Vamos criar um tipo de dados mais complexo --> uma lista de tuplas.
```

```
estudantes = [('Alice', 'Brown'), ('Jack', 'White'), ('Molly', 'Green')]
pd.Series(estudantes)
```

[5]:
```
0    (Alice, Brown)
1     (Jack, White)
2    (Molly, Green)
dtype: object
```

Vemos que cada uma das tuplas é armazenada no objeto da série e o tipo é do tipo object.

[6]:
```
# Também podemos separar a criação do índice dos dados passando um índice como␣
↪uma lista explicitamente para a série.

s = pd.Series(['Física', 'Química', 'Inglês'], index = ['Alice', 'Jack',␣
↪'Molly'])
s
```

[6]:
```
Alice     Física
Jack     Química
Molly     Inglês
dtype: str
```

[7]:
```
# O que acontece se a lista de valores no objeto de índice não estiver alinhada␣
↪com as chaves em seu dicionário para criar a série??
# O Pandas substitui a criação automática para favorecer somente os valores de␣
↪índice que foi fornecido. Portanto, será ignorado do dicionário todas as␣
↪chaves que não estão no seu índice e o Pandas adicionará NaN.

# Vamos passar um dicionário de três itens, com alunos e seus cursos
curso_estudante = {'Alice': 'Física',
                   'Jack': 'Química',
                   'Molly': 'Inglês'}

# Quando criar a serie, vamos pedir apenas um índice com três alunos e␣
↪excluirei Jack.
s = pd.Series(curso_estudante, index = ['Alice', 'Molly', 'Sam'])
s
```

[7]:
```
Alice    Física
Molly    Inglês
Sam         NaN
dtype: str
```

16_consultando_serie

February 27, 2026

1 Consultando uma série

É importante pensar em paralelização ao se envolver na programação de ciência de dados

[1]:
```
import pandas as pd
```

[2]:
```
# Uma série Pandas pode ser consultada pela posição do índice ou pelo rótulo do␣
↪índice. Se não for fornecido um índice para a série durante a consulta, a␣
↪posição e o rótulo são os mesmos valores.
# Para consultar por localização numérica, começando em 0, use o atributo iloc.␣
↪Para consultar pelo rótulo do índice, pode utilizar o atributo loc.

# Vamos utilizar alunos matriculados em aulas provenientes de um dicionário.
aulas_estudantes = {'Alice': 'Física',
                    'Jack': 'Química',
                    'Molly': 'Inglês',
                    'Sam': 'História'}

s = pd.Series(aulas_estudantes)
s
```

[2]:
```
Alice      Física
Jack      Química
Molly      Inglês
Sam      História
dtype: str
```

[3]:
```
# Para esta série, se quisermos ver a quarta entrada, podemos usar o atributo␣
↪iloc com o parâmetro 3.
s.iloc[3]
```

[3]: 'História'

[4]:
```
# Se quisermos ver qual classe Molly tem, usaríamos o atributo loc com um␣
↪parâmetro de molly.
s.loc['Molly']
```

[4]: 'Inglês'

Mas qual a diferença entre: - s.loc[‘Molly’] - s.Molly

A diferença é como o Pandas está acessando o dado. A forma s.loc é a forma mais segura, funciona sempre, com qualquer nome, mesmo se tiver espaço, número, etc.

Agora quando temos um s.atributo, o Pandas permite isso apenas quando o nome do índice é um identificador válido de Python, isto é, não pode ter espaço, não pode começar com número, e não pode ter nome de métodos, como mean.

[8]:
```
# Perceba que não podemos acessar um valor pelo método de indexação, como s[0],␣
↪apenas se o rótulo for 0

estudantes = ['Alice,', 'Jack', 'Molly']
serie = pd.Series(estudantes)
serie[0]
```

[8]: 'Alice,'

[15]:
```
# O que acontece se seu índice for uma lista de números inteiros?
# O Pandas não pode determinar automaticamente se você pretende consultar por␣
↪posição de índice ou rótulo de índice, então é necessário ter cuidado ao␣
↪usar, ou seja, a opção mais segura é ser bem explícito (utilizando iloc e␣
↪loc).

codigo_classe = {99: 'Fisica', 100: 'Química', 101: 'Inglês', 102: 'História'}
s = pd.Series(codigo_classe)
print(s[99])
print(s.loc[99])
print(s.iloc[0])
```

```
Fisica
Fisica
Fisica
```

17_operacoes_vetoriais

February 27, 2026

1 Operações Vetoriais

Uma tarefa comum é querer considerar todos os valores dentro de uma série e fazer algum tipo de operação. Isso pode ser tentar encontrar um determinado número, resumir os dados ou transformá-los de alguma forma.

[1]:
```
# Biblioteca
import pandas as pd
```

[2]:
```
# Uma abordagem programática típica para isso seria repetir todos os itens da␣
↪série e invocar a operação na qual estamos interessados.
# Por exemplo, poderíamos criar uma série de números inteiros reprexentando as␣
↪notas dos alunos e tentar obter a média.

notas = pd.Series([9, 8, 7, 6])
total = 0

total = sum(nota for nota in notas) / len(notas)

total
```

[2]: 7.5

[3]:
```
# Isso pode funcionar, mas podemos aproveitar mais do hardware. O Pandas␣
↪suporta métodos de computação vetorial. Para isso, precisamos da biblioteca␣
↪NumPy.

import numpy as np

# Agora chamaremos np.sum() e passaremos um iterável, isto é, nesse caso, nossa␣
↪série do Pandas.
total = np.sum(notas) / len(notas)
total
```

[3]: np.float64(7.5)

Os dois métodos mostrados produzem o mesmo resultado, mas a diferença é que com o método np.sum() estaremos utilizando operações vetoriais (uso de registradores vetoriais (SIMD) e é im-

plementado em C), mas isso não significa que é garatido o uso de SIMD, mas o importante é que diminui o overhead do Python. Além disso, como são utilizados arrays do numpy, há possibilidade de termos o uso de memória contígua. Enquanto o sum() do próprio Python percorre elemento por elemento somando de forma iterativa

[17]:
```
# Vamos criar uma grande série de números aleatórios para provar se np.sum() é␣
↪mais rápido que sum()
numeros = pd.Series(np.random.rand(1000000))

# Vamos olhar os cinco primeiros itens desta série para garantir que eles␣
↪existem
print(numeros.head())
print(len(numeros))
```

```
0    0.634393
1    0.517234
2    0.380207
3    0.604367
4    0.656448
dtype: float64
1000000
```

Agora que temos uma grande série. O interpretador do Python tem magic functions do Jupyter/IPython que começam com '%%'. Então vamos usar uma função mágica celular chamada 'timeit', em que essa função executará nosso código algumas vezes para determinar, em média, quanto tempo leva para executar a função.

OBS: Podemos dar ao timeit o número de loops que gostaríamos de executar, mas por padrão são 1000 loops. OBS2: Para utilizar a função cellular magic, ela precisa ser a primeira linha da célula.

As magic functions são comandos especiais do Jupyter/IPython que começam com % (linha) ou %% (célula). Elas não fazem parte do Python padrão, mas são recursos do ambiente interativo para facilitar tarefas como medir tempo, executar comandos do sistema, instalar pacotes e rodar código em outras linguagens.

- % → atua em uma única linha (ex: %time, %ls, %pwd)
- %% → atua em toda a célula (ex: %%time, %%bash)

Exemplo: %time sum(range(1000000)) %%time sum(range(1000000))

Exemplo: %%timeit -n NUM_EXECUCOES -r NUM_REPETICOES_POR_EXECUCAO

[19]:
```
%%timeit -n 100

total = sum(numero for numero in numeros)
total / len(numeros)
```

```
94.1 ms ± 887  s per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

[25]:
```
%%timeit -n 100
```

```
total = np.sum(numeros)
total / len(numeros)
```

```
1.14 ms ± 81.4  s per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Esse resultado vem do uso de computação paralela.

[26]:
```
# Um recurso relacionado no Pandas e no NumPy é chamado de broadcasting. Com␣
↪broadcasting, podemos aplicar uma operação a cada valor da serie, alterando␣
↪a série de forma muito rápida (espalhando para todos os itens da série).
# Por exemplo, se quiséssemos aumentar cada variável aleatória em 2, poderíamos␣
↪fazer isso rapidamente usando o operador += no objeto da série.

# Broadcasting permite operações entre:
# Vetor + Escalar
# Vetor + Vetor de mesmo tamanho
# Matriz + Vetor
# Matriz + Escalar
# OBS: Desde que as dimensões sejam compatíveis.

# Olhando as 5 primeiras linhas da série
numeros.head()
```

[26]:
```
0    0.634393
1    0.517234
2    0.380207
3    0.604367
4    0.656448
dtype: float64
```

[27]:
```
# Agora vamos aumentar tudo na série em 2 unidades

numeros += 2
numeros.head()
```

[27]:
```
0    2.634393
1    2.517234
2    2.380207
3    2.604367
4    2.656448
dtype: float64
```

Aqui também tem o uso de SIMD e de memória contígua se as condições permitirem.

O que são condições que não permitem o uso de vetorização? - A SIMD funciona melhor com tipos numéricos simples, como int e float, mas se for object, então não tem como utilizar SIMD. - Se os dados não estiverem em memória contígua, fica difícil carregá-los para o registrador vetorial. - SIMD funciona para operações simples, como +-/* - E obviamente, a CPU precisa ter suporte para operações vetoriais.

[37]:
```
# SE FOSSEMOS FAZER DE FORMA ITERATIVA, SERIA:
# Pandas suporta a iteração da série, assim como em um dicionário, o que nos␣
↪permite descompactar valores facilmente.

# Vamos utilizar a função iteritems(), o que retorna um rótulo e um valor.
for label, valor in numeros.items():
    # O item que é retornado, vamos chamar de set_value()
    numeros.loc[label] = valor + 2

numeros.head()
```

[37]:
```
0    4.634393
1    4.517234
2    4.380207
3    4.604367
4    4.656448
dtype: float64
```

ATENÇÃO: Se você estiver iterando a qualquer momento no Pandas ou NumPy, se pergunte se está fazendo as coisas da melhor maneira possível

OBS: Podemos criar nossas próprias funções vetorizadas

18_anexando_series

February 27, 2026

1 Anexando Séries

Uma última observação sobre o uso dos operadores de indexação para acessar os dados da série. O atributo .loc permite não apenas modificar os dados no local, mas também adicionar novos dados, isto é, se o valor que inserimos como índice não existir, uma nova entrada será criada, como no dicionário.

[3]:
```
import pandas as pd
```

[6]:
```
# Vamos a um exemplo usando uma série de algus números
s = pd.Series([1, 2, 3])
print(s)

# Vamos adicionar um novo valor
s.loc['Historia'] = 102
print(s)
```

```
0    1
1    2
2    3
dtype: int64
0             1
1             2
2             3
Historia    102
dtype: int64
```

[8]:
```
# Até agora foi mostrado apenas exemplos de uma série em que os valores do␣
↪índice eram exclusivos (unique). Vamos a um exemplo em que os valores do␣
↪índice não são exclusivos.

# Vamos criar uma série de estudantes e cursos que eles fizeram
aulas_estudantes = pd.Series({'Alice': 'Física',
                    'Jack': 'Química',
                    'Molly': 'Inglês',
                    'Sam': 'História'})

aulas_estudantes
```

[8]:
```
Alice      Física
Jack      Química
Molly      Inglês
Sam      História
dtype: str
```

[9]:
```python
# Agora, vamos criar uma série só para a nova aluna Kelly, na qual lista todos␣
↪os cursos que ela fez. Definiremos o índice como Kelly e os dados como os␣
↪nomes dos cursos.

aulas_kelly = pd.Series(['Filosofia', 'Artes', 'Matemática'], index = ['Kelly',␣
↪'Kelly', 'Kelly'])
aulas_kelly
```

[9]:
```
Kelly     Filosofia
Kelly         Artes
Kelly    Matemática
dtype: str
```

[ ]:
```python
# Finalmente, podemos anexar todos os dados dessa nova série à primeira usando␣
↪a função .concat(). Isso criará alguma variável em todas as classes dos␣
↪alunos.

todos_estudantes = pd.concat([aulas_estudantes, aulas_kelly])

# Isso cria uma série que inclui nosso pessoal original bem como todos os␣
↪cursos da Kelly.
todos_estudantes
```

[ ]:
```
Alice        Física
Jack        Química
Molly        Inglês
Sam        História
Kelly     Filosofia
Kelly         Artes
Kelly    Matemática
dtype: str
```

Isso apenas juntou as duas séries em uma nova sem alterar as antigas

[ ]:
```python
# Agora, como não temos mais um index exclusivo (Kelly), podemos ver que quando␣
↪consultamos a série anexada para o index Kelly não teremos mais um único␣
↪valor, teremos uma nova série.

todos_estudantes.loc['Kelly']
```

```
[ ]: Kelly     Filosofia
     Kelly         Artes
     Kelly    Matemática
     dtype: str
```

19_data_frame

February 27, 2026

1 Estrutura de dados DataFrame

O DataFrame é uma estrutura de dados que é o coração da biblioteca Pandas, muito usado em tarefas de filtragem. O DataFrame é conceitualmente um objeto bidimensional, em que há um índice e varias colunas de conteúdo com cada coluna possuindo um rótulo.

[130]:
```
# Importando a biblioteca
import pandas as pd
```

[131]:
```
# Vamos criar três registros escolares paraestudantes e suas notas. Criarei uma␣
↪série para cada nome de estudante, o nome da matéria e a pontuação.

registro1 = pd.Series({'Nome': 'Alice',
                       'Curso': 'Física',
                       'Nota': 8.5})

registro2 = pd.Series({'Nome': 'Jack',
                       'Curso': 'Química',
                       'Nota': 8.2})

registro3 = pd.Series({'Nome': 'Helen',
                       'Curso': 'Biologia',
                       'Nota': 9})
```

[132]:
```
# Como uma série, o DataFrame é indexado. Usaremos um grupo de séries em que␣
↪cada série representa uma linha de dados.
# Assim como as funções de séries, podemos passar os nossos itens idividuais em␣
↪um array e podemos passar os nomes dos índices.

# Criando um DataFrame
df = pd.DataFrame([registro1, registro2, registro3], index = ['aluno1',␣
↪'aluno2', 'aluno3'])

# Vamos olhar as 5 primeiras linhas
df.head()
```

[132]:

```
         Nome     Curso  Nota
aluno1  Alice    Física   8.5
aluno2   Jack   Química   8.2
aluno3  Helen  Biologia   9.0
```

[133]:

```
# Um método alternativo seria usar uma lista de dicionários, em que cada␣
↪dicionário representa uma l  inha de dados.

estudantes = [{'Nome': 'Alice',
               'Curso': 'Física',
               'Nota': 8.5},
                {'Nome': 'Jack',
                'Curso': 'Química',
                'Nota': 8.2},
                {'Nome': 'Helen',
                'Curso': 'Biologia',
                'Nota': 9}]

# Agora passamos essa lista de dicionários para o DataFrame
df = pd.DataFrame(estudantes, index = ['aluno1', 'aluno2', 'aluno3'])
df
```

[133]:

```
         Nome     Curso  Nota
aluno1  Alice    Física   8.5
aluno2   Jack   Química   8.2
aluno3  Helen  Biologia   9.0
```

[134]:

```
# Podemos extrair dados usando os atributos 'iloc' e 'loc'.
# Dado que o DataFrame é bidimensional, passando um único valor para o operador␣
↪de indexação loc, iremos retornar uma série se houver apenas uma linha de␣
↪retorno.

# Por exemplo, se quisermos selecionar dados associados ao aluno2 apenas␣
↪consultamo o atributo '.loc' com um único parâmetro.
df.loc['aluno2']
```

[134]:

```
Nome        Jack
Curso    Química
Nota         8.2
Name: aluno2, dtype: object
```

Perceba que o nome da série é apresendado como valor do índice (o nome da linha), enquanto que o nome da coluna é incluído na saída

[135]:

```
# Podemos verificar o tipo de dado da resposta usando a função 'type()' do␣
↪Python.
```

```
type(df.loc['aluno2'])
```

[135]: pandas.Series

[136]:
```
# É importante lembrar que os índices e os nomes das colunas ao longo dos eixos␣
↪horizontal e vertical podem não ser únicos (unique).

alunos_escola = [{'Nome': 'Alice',
               'Curso': 'Física',
               'Nota': 8.5},
                {'Nome': 'Jack',
                'Curso': 'Química',
                'Nota': 8.2},
                {'Nome': 'Helen',
                'Curso': 'Biologia',
                'Nota': 9}]

df = pd.DataFrame(estudantes, index = ['escola1', 'escola2', 'escola1'])

# Nesse exemplo, vamos ver dois registros para o 'escola1' como linhas␣
↪diferentes.
# Neste caso, como o nome da linha não é único, vamos ter, não uma série nova,␣
↪mas sim um DataFrame novo
df.loc['escola1']
```

[136]:

	Nome	Curso	Nota
escola1	Alice	Física	8.5
escola1	Helen	Biologia	9.0

[137]:
```
# Aqui podemos ver o tipo de dados diferente.

type(df.loc['escola1'])
```

[137]: pandas.DataFrame

[138]:
```
# Um dos poderes do DataFrame do Pandas é que conseguimos selecionar␣
↪rapidamente dados baseados em vários eixos.
# Por exemplo, se quisermos apenas listar os nomes dos alunos da 'escola1',␣
↪podemos fornecer dois parâmetros para '.loc', sendo um o índice de linha (o␣
↪nome da linha) e o outro sendo o nome da coluna.

# Vamos supor que estamos interessados em alguns estudantes da 'escola1'
df.loc['escola1', 'Nome']
```

[138]:
```
escola1    Alice
escola1    Helen
Name: Nome, dtype: str
```

Perceba que isso é uma série do Pandas

[139]:
```
df.loc['escola1', ['Nome', 'Nota']]
```

[139]:
```
          Nome  Nota
escola1  Alice   8.5
escola1  Helen   9.0
```

Aqui já é um DataFrame. Isso, pois temos duas colunas de atributos

[140]:
```
# O que faríamos se quiséssemos que ele selecionasse linhas de coluna única?
# Existem alguns mecanismos. Primeiro, fazemos a transposição da matriz, o que␣
↪transforma todas as linhas em colunas e todas as colunas em linhas, e isso é␣
↪feito com o atributo 'T'.

df.T
```

[140]:
```
       escola1  escola2   escola1
Nome     Alice     Jack     Helen
Curso   Física  Química  Biologia
Nota       8.5      8.2       9.0
```

[141]:
```
# Para usarmos '.loc' na matriz transposta para obter apenas os nomes dos␣
↪estudantes faremos:

df.T.loc['Nome']
```

[141]:
```
escola1    Alice
escola2     Jack
escola1    Helen
Name: Nome, dtype: object
```

ATENÇÃO: Tanto .loc quanto .iloc são usados para selecionar linhas. OBS: Use a transposta apenas se necessariamente precisar, pois tem métodos melhores para selecionar apenas colunas específicas com menor custo.

[142]:
```
# Pandas disponibiliza o operador de indexação diretamente no DataFrame para␣
↪selecionar a coluna. Em DataFrames as colunas SEMPRE possuem um nome (um␣
↪título), e, assim, a seleção será sempre baseada no seu nome.

df['Nome']
```

[142]:
```
escola1    Alice
escola2     Jack
escola1    Helen
Name: Nome, dtype: str
```

[143]:
```
# Para acessar duas colunas:
df[['Nome', 'Curso']]
```

[143]:
```
          Nome     Curso
escola1  Alice    Física
escola2   Jack   Química
escola1  Helen  Biologia
```

[144]:
```
# Na prática, isso funciona muito bem, uma vez que estamos frequentemente␣
↪tentando adicionar ou remover colunas. No entanto, isso signfica que podemos␣
↪receber um erro de chave quando tentamos usar o .loc com um nome de coluna.

'''df.loc['Nome']'''          # ERRO
```

[144]: "df.loc['Nome']"

[145]:
```
# Note também que o resultado de uma única projeção de coluna é um objeto do␣
↪tipo Série

type(df['Nome'])
```

[145]: pandas.Series

[146]:
```
# Uma vez que o resultado do operador de indexação é um DataFrame ou uma Série,␣
↪podemos usar essas operações juntas. Por exemplo, podemos selecionar todas␣
↪as linhas que são relacionadas com 'escola1' usando .loc, e obtemos pelo␣
↪nome da coluna apenas para essas linhas.

df.loc['escola1']['Nome']
```

[146]:
```
escola1    Alice
escola1    Helen
Name: Nome, dtype: str
```

[147]:
```
# Para mais de uma coluna:
df.loc['escola1'][['Nome', 'Curso']]
```

[147]:
```
          Nome     Curso
escola1  Alice    Física
escola1  Helen  Biologia
```

Mas qual é a diferença entre:

- df.loc[‘escola1’, ‘Nome’]
- df.loc[‘escola1’, [‘Nome’]]
- df.loc[‘escola1’][‘Nome’]

A PRINCIPAL diferença está na eficiência. No primeiro caso (df.loc[‘escola1’, ‘Nome’]), é acessado

a linha escola1 e a coluna nome DIRETO. No segundo caso (df.loc['escola1', ['Nome']]), acontece a mesma coisa, porém como é passado em forma de lista as colunas, então ao invés de formatar com Série é formatado com DataFrame. Ja no terceiro caso (df.loc['escola1']['Nome']), primeiro é acessado a linha, de forma inteira, depois dessa linha(s), filtra-se a coluna 'Nome'. Nesse caso, temos duas operações, então é menos eficiente.

Sendo assim, a terceira opção, isto é, encadeando por indexação no retorno da função de outro índice, possui custo, pois ele quebra a operação em duas etapas separadas, então é melhor evitar se você puder fazer de outra forma. Particularmente, o encadeamento tente a fazer com que o Pandas retorne uma cópia do DataFrame (ou uma cópia de parte do DataFrame) ao invés de trazer uma 'view' do DataFrame. Dessa forma, para selecionar dados, essa não é uma boa opção.

[148]:
```
# Uma outra abordagem é: à medida em que vemos que .loc faz a seleção da linha,␣
↪e pode ter dois parâmetros, o índice da linha e uma lista dos nomes das␣
↪colunas, o .loc também suporta fatiar (operação de fatiamento).

# Se quisermos selecionar todas as linhas, podemos usar dois pontos para␣
↪indicar todos os registros do começo ao fim. Isso é como fatiar caracteres␣
↪em uma lista em Python.
# Em seguida, podemos adicionar o nome da coluna como o segundo parâmetro em␣
↪uma str, e se quiséssemos incluir várias colunas, podemos fazer na forma de␣
↪lista e teremos como retorno somente as colunas que pedimos.

# Por exemplo, no qual pedimos todos os nomes e as notas de todas as escolas␣
↪usando o operador .loc
df.loc[:, ['Nome', 'Nota']]
```

[148]:

	Nome	Nota
escola1	Alice	8.5
escola2	Jack	8.2
escola1	Helen	9.0

Entre: - df.loc[:, ['Nome', 'Nota']] - df[['Nome', 'Nota']]

Não há diferença, apenas na escrita mesmo

[149]:
```
# Vamos ver como deletar dados de Séries e de DataFrames, e podemos usar a␣
↪função 'drop()' para isso.
# Essa função leva um único parâmetro, que é o índice/rótulo de linha a remover.
↪
# ATENÇÃO: A função drop() não altera o DataFrame original, em vez disso, a␣
↪função drop() retorna uma cópia do DataFrame com as linhas informadas␣
↪removidas

df.drop('escola1')
```

[149]:

	Nome	Curso	Nota
escola2	Jack	Química	8.2

[150]:
```python
# Agora vamos olhar o original:
df
```

[150]:
```
          Nome     Curso  Nota
escola1  Alice    Física   8.5
escola2   Jack   Química   8.2
escola1  Helen  Biologia   9.0
```

Perceba que ainda está igual o original

[151]:
```python
# A função drop() tem dois parâmetros opcionais.
# - O primeiro é chamado "inplace", e se estiver definido como verdadeiro, ai␣
↪sim o DataFrame original será modificado ao invés de retornar uma cópia
# - O segundo é chamado "axis", definindo qual eixo será descartado, e, por␣
↪padrão, o seu valor é ZERO (0), indicando o eixo da LINHA. Porém, podemos␣
↪mudar para UM (1) se quisermos descartar a coluna.

copia_df = df.copy()

# Agora vamos remover uma coluna
copia_df.drop('Nota', axis = 1, inplace = True)
copia_df
```

[151]:
```
          Nome     Curso
escola1  Alice    Física
escola2   Jack   Química
escola1  Helen  Biologia
```

Perceba que a coluna NOTA foi removida, e de forma definitiva

[152]:
```python
# Há, no entanto, uma outra forma de descartar colunas, e isso é diretamente␣
↪através do uso de um operador de indexação usando a palavra chave 'del'.␣
↪Dessa forma, estaremos removendo dados, no entanto, produz efeitos imediatos␣
↪sobre o DataFrame e não apresenta mais os dados removidos.
del copia_df['Curso']
copia_df
```

[152]:
```
          Nome
escola1  Alice
escola2   Jack
escola1  Helen
```

[153]:
```python
# Agora vamos remover uma linha
copia_df.drop('escola1', inplace = True)
copia_df
```

[153]:
```
         Nome
escola2  Jack
```

CUIDADO: Perceba que foi deletadas TODAS as linhas que tinham o mesmo nome 'escola1'

[155]:
```
import numpy as np
```

[157]:
```
# Podemos adicionar uma nova coluna ao DataFrame, basta usar um valor usando o␣
↪operador de indexação, como no dicionário.
# Por exemplo, se quisermos adicionar uma coluna 'ClassRanking' com o valor␣
↪default igual 'NaN', podemos fazê-lo usando o operador de atribuição.

copia_df['ClassRanking'] = np.nan
copia_df
```

[157]:

	Nome	ClassRanking
escola2	Jack	NaN

Todas as linhas ficam com o valor padrão

20_indexacao_carregamento_dataframe

February 27, 2026

1 Indexação e carregamento de DataFrame

Um fluxo de trabalho comum é ler o conjunto de dados, geralmente de um arquivo externo e então começar a limpar e manipular o conjunto de dados para analise. Vamos aprender como carregar dados .CSV em um DataFrame.

OBS: .CSV é um tipo de arquivo leve

[2]:
```
import pandas as pd
```

[3]:
```
# OBS: o Jupyter Notebook usa iPython como um kernel subjacente que fornece␣
↪caminhos convenientes de integrar comandos shell de baixo nível que são␣
↪programas executados no SO. Isso é muito útil para a integração de seus␣
↪fluxos de trabalho em ciência de dados.
# Vamos utilizar o comando 'cat' para 'concatenar', o qual exibe o conteúdo de␣
↪um arquivo.
# No iPython, se colocarmos um ponto de exclamação no início da linha, ele␣
↪executará o restante da linha como um comando de terminal

!cat datasets/Admission_Predict.csv
```

```
Serial No.,GRE Score,TOEFL Score,University Rating,SOP,LOR ,CGPA,Research,Chance
of Admit
1,337,118,4,4.5,4.5,9.65,1,0.92
2,324,107,4,4,4.5,8.87,1,0.76
3,316,104,3,3,3.5,8,1,0.72
4,322,110,3,3.5,2.5,8.67,1,0.8
5,314,103,2,2,3,8.21,0,0.65
6,330,115,5,4.5,3,9.34,1,0.9
7,321,109,3,3,4,8.2,1,0.75
8,308,101,2,3,4,7.9,0,0.68
9,302,102,1,2,1.5,8,0,0.5
10,323,108,3,3.5,3,8.6,0,0.45
11,325,106,3,3.5,4,8.4,1,0.52
12,327,111,4,4,4.5,9,1,0.84
13,328,112,4,4,4.5,9.1,1,0.78
14,307,109,3,4,3,8,1,0.62
15,311,104,3,3.5,2,8.2,1,0.61
```

```
400,333,117,4,5,4,9.66,1,0.95
```

Vemos, no resultado, que há uma lista de colunas e que os identificadores da coluna estão listados em str na primeira linha do arquivo. Depois temos as linhas de dados, todas as colunas estão separadas por vírgulas.

[4]:
```
# O Pandas facilita a transformação de um CSV em um DataFrame. Apenas chamamos␣
↪a função read_csv()

df = pd.read_csv('datasets/Admission_Predict.csv')

# Vamos ver as 5 primeiras linhas
df.head()
```

[4]:
```
   Serial No.  GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
0           1        337          118                  4  4.5   4.5  9.65
1           2        324          107                  4  4.0   4.5  8.87
2           3        316          104                  3  3.0   3.5  8.00
3           4        322          110                  3  3.5   2.5  8.67
4           5        314          103                  2  2.0   3.0  8.21

   Research  Chance of Admit
0         1             0.92
1         1             0.76
2         1             0.72
3         1             0.80
4         0             0.65
```

Note que, por padrão, o Pandas criou no DataFrame um índice para as linhas, porém, nesse caso, o CSV tem índice, por exemplo, o Serial No.

[ ]:
```
# Vamos definir o 'Serial No.' como índice
df = pd.read_csv('datasets/Admission_Predict.csv', index_col = 'Serial No.')

# Também poderíamos passar o índice da coluna, por exemplo, index_col = 0.␣
↪Daria no mesmo.
df.head()
```

[ ]:
```
            GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
Serial No.
1                 337          118                  4  4.5   4.5  9.65
2                 324          107                  4  4.0   4.5  8.87
3                 316          104                  3  3.0   3.5  8.00
4                 322          110                  3  3.5   2.5  8.67
5                 314          103                  2  2.0   3.0  8.21

            Research  Chance of Admit
Serial No.
```

```
1                   1              0.92
2                   1              0.76
3                   1              0.72
4                   1              0.80
5                   0              0.65
```

[14]:
```
# Perceba também que temos algumas colunas, como SOP e LOR, que possuem siglas,␣
↪e, provavelmente, nem todos sabem o que essas siglas signficam. Portanto,␣
↪vamos alterar os nomes dessas colunas para torná-los mais claros.
# Podemos usar a função rename() do Pandas.

# Alterando o nome de uma coluna
novo_df = df.rename(columns = {'GRE Score': 'GRE Score', 'TOEFL Score': 'TOEFL␣
↪Score', 'University Rating': 'University Rating', 'SOP': 'Statement of␣
↪Pupose', 'LOR': 'Letter of Recommendation', 'CGPA': 'CGPA', 'Research':␣
↪'Research', 'Chance of Admit': 'Chance of Admit'})

novo_df.head()
```

[14]:
```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1                 337          118                  4                  4.5
2                 324          107                  4                  4.0
3                 316          104                  3                  3.0
4                 322          110                  3                  3.5
5                 314          103                  2                  2.0

            LOR   CGPA  Research  Chance of Admit
Serial No.
1            4.5  9.65         1             0.92
2            4.5  8.87         1             0.76
3            3.5  8.00         1             0.72
4            2.5  8.67         1             0.80
5            3.0  8.21         0             0.65
```

[ ]:
```
# Pelo resultado, nota-se que apenas o 'SOP' mudou, mas o 'LOR' não mudou, por␣
↪quê?

# Vamos investigar um pouco, então PRIMEIRO, temos que ter certeza de que TODOS␣
↪os nomes das colunas que serão renomeadas estão corretos.
novo_df.columns
```

[ ]:
```
Index(['GRE Score', 'TOEFL Score', 'University Rating', 'Statement of Pupose',
       'LOR ', 'CGPA', 'Research', 'Chance of Admit '],
      dtype='str')
```

Olhe atentamente e veja que a coluna com o rótulo ‘LOR’ possui um espaço bem discreto após a

o ‘R’ de ‘LOR’. Também ocorre o mesmo com ‘Chance of Admit’.

É por isso que nosso dicionário renomeado não funciona para ‘LOR’

[21]:
```
# Há algumas maneiras de resolver isso
# A primeira é incluindo o espaço no novo nome

novo_df = df.rename(columns = {'SOP': 'Statement of Pupose', 'LOR ': 'Letter of␣
↪Recommendation'})

# OBS: NÃO PRECISA PASSAR TODOS OS NOMES, BASTA PASSAR APENAS O QUE QUER MUDAR,␣
↪e nem precisa estar na mesma ordem

novo_df.head()
```

[21]:
```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1                 337          118                  4                  4.5
2                 324          107                  4                  4.0
3                 316          104                  3                  3.0
4                 322          110                  3                  3.5
5                 314          103                  2                  2.0

            Letter of Recommendation  CGPA  Research  Chance of Admit
Serial No.
1                                4.5  9.65         1             0.92
2                                4.5  8.87         1             0.76
3                                3.5  8.00         1             0.72
4                                2.5  8.67         1             0.80
5                                3.0  8.21         0             0.65
```

[27]:
```
# Isso funciona, mas é frágil, pois pense se for um 'TAB', como saber quantos␣
↪espaços? dois? quatro?

# Outra maneira é uma função que faça a limpeza e, em seguida, dizer ao␣
↪"rename" para aplicar essa função a todos os dados. Vamos usar a função .
↪strip() para tirar os espaços inúteis.
# Quando passamos isso para o "rename", passamos a função como o parâmetro do␣
↪"mapper" e, depois, indicamos se o eixo deve ser coluna ou índice, ou seja,␣
↪os rótulos das linhas.

novo_df = df.rename(mapper = str.strip, axis = 1)

novo_df.columns
# OBS: No axis, poderíamos colocar axix = 'columns' ao invés do número 1␣
↪indicando a coluna e o 0 a linha
```

[27]:
```
Index(['GRE Score', 'TOEFL Score', 'University Rating', 'SOP', 'LOR', 'CGPA',
       'Research', 'Chance of Admit'],
      dtype='str')
```

Perceba que agora, 'LOR' e 'Chance of Admit' não possuem mais espaços inúteis.

[34]:
```
# novo_df = df.rename(mapper = str.strip, axis = 1, columns = {'SOP':␣
↪'Statement of Pupose', 'LOR': 'Letter of Recommendation'}) --> ERRO!!!

# axis=1 → "renomeie as colunas"
# columns={...} → "renomeie as colunas"

# Agora podemos renomear do jeito certo
novo_df = novo_df.rename(columns = {'SOP': 'Statement of Pupose', 'LOR':␣
↪'Letter of Recommendation'})

novo_df.head()
```

[34]:

	GRE Score	TOEFL Score	University Rating	Statement of Pupose	\
Serial No.					
1	337	118	4	4.5	
2	324	107	4	4.0	
3	316	104	3	3.0	
4	322	110	3	3.5	
5	314	103	2	2.0	

	Letter of Recommendation	CGPA	Research	Chance of Admit
Serial No.				
1	4.5	9.65	1	0.92
2	4.5	8.87	1	0.76
3	3.5	8.00	1	0.72
4	2.5	8.67	1	0.80
5	3.0	8.21	0	0.65

ATENÇÃO: Esses espaços em branco que são 'invisíveis' são muito comuns, então como boa prática, sempre que importar e for renomear alguma coluna, utilizar o mapper antes.

[38]:
```
# Também podemos usar o atributo df.columns. Atribuindo-o a uma lista de nomes␣
↪de coluna que renomeará diretamente as colunas. Isso modificará diretamente␣
↪o DataFrame original, sem criar uma cópia como estamos fazendo, e é muito␣
↪eficiente, principalmente quando temos muitas colunas e queremos renomear␣
↪apenas algumas delas.
# Além disso, essa nova técnica também NÃO é afetada por erros sutis nos nomes␣
↪das colunas, como o problema que acabamos de encontrar.

# Por exemplo, vamos alterar todos os valores dos nomes das colunas para letras␣
↪minúsculas. Então, primeiro, precisamos obter nossas listas.
```

```python
colunas = list(df.columns)

# Agora, vamos escrever uma pequena lista de compreensão (é o que será feito␣
↪nas colunas do DataFrame, mas antes, será feito na nossa lista)
colunas = [x.lower().strip() for x in colunas]
# Isso vai escrever todos os atributos de .columns em mínusculo, além de tirar␣
↪espaços inúteis

# Agora, vamos passar a mudança para as colunas do DataFrame
df.columns = colunas
df.head()
```

[38]:

```
            gre score  toefl score  university rating  sop  lor  cgpa  \
Serial No.
1                 337          118                  4  4.5  4.5  9.65
2                 324          107                  4  4.0  4.5  8.87
3                 316          104                  3  3.0  3.5  8.00
4                 322          110                  3  3.5  2.5  8.67
5                 314          103                  2  2.0  3.0  8.21

            research  chance of admit
Serial No.
1                  1             0.92
2                  1             0.76
3                  1             0.72
4                  1             0.80
5                  0             0.65
```

21_consulta_dataframe

February 27, 2026

1 Consulta a um DataFrame

Vamos entender como consultar DataFrames. A primeira etapa do processo é entender a máscara booleana. Um mascaramento booleano é a essência da primeira consulta rápida e eficiente tanto no NumPy quanto no Pandas.

LEMBRANDO: Uma máscara booleana é um array/matriz em que cada um dos valores do array é verdadeiro ou falso. Essa matriz é essencialmente sobreposta à outra estrutura de dados que estamos consultando, e qualquer célula alinhada com o valor verdadeiro será passada como resultado.

[23]:
```
# Importando a biblioteca
import pandas as pd
```

[24]:
```
# Vamos carregar o arquivo CSV
df = pd.read_csv('datasets/Admission_Predict.csv', index_col = 0)
df.head()
```

[24]:
```
            GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
Serial No.
1                 337          118                  4  4.5   4.5  9.65
2                 324          107                  4  4.0   4.5  8.87
3                 316          104                  3  3.0   3.5  8.00
4                 322          110                  3  3.5   2.5  8.67
5                 314          103                  2  2.0   3.0  8.21

            Research  Chance of Admit
Serial No.
1                  1             0.92
2                  1             0.76
3                  1             0.72
4                  1             0.80
5                  0             0.65
```

[25]:
```
# Vamos limpar e corrigir colunas mal nomeadas com compreensão de listas
colunas = list(df.columns)
colunas = [x.strip() for x in colunas]

colunas[3] = 'Statement of Pupose'
```

```
colunas[4] = 'Letter of Recommendation'

df.columns = colunas
df.head()
```

[25]:

```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1                 337          118                  4                  4.5
2                 324          107                  4                  4.0
3                 316          104                  3                  3.0
4                 322          110                  3                  3.5
5                 314          103                  2                  2.0

            Letter of Recommendation  CGPA  Research  Chance of Admit
Serial No.
1                                4.5  9.65         1             0.92
2                                4.5  8.87         1             0.76
3                                3.5  8.00         1             0.72
4                                2.5  8.67         1             0.80
5                                3.0  8.21         0             0.65
```

[26]:

```
# As máscaras booleanas são criadas aplicando os operadores diretamente aos␣
↪objetos do Pandas, isto é, Séries ou DataFrames.

# Por exemplo, estamos interessados em ver apenas os alunos que têm chances de␣
↪admissão maior que 0.7.
mascara_booleana = df['Chance of Admit'] > 0.7
mascara_booleana
```

[26]:

```
Serial No.
1       True
2       True
3       True
4       True
5      False
       …
396     True
397     True
398     True
399    False
400     True
Name: Chance of Admit, Length: 400, dtype: bool
```

Perceba que o resultado do broadcasting de um operador de comparação é uma máscara booleana, com valores verdadeiros ou falsos, dependendo dos resultados da comparação. Por baixo, o Pandas está aplicando o operador de comparação que foi especificado por meio de vetorização, portanto, ocorre em paralelo e de forma eficiente.

OBS: Não em (df[['Chance of Admit']] > 0.7, isto é, com [] a mais para passar no formato de DataFrame e não de Série, pois o resultado que será considerado será sempre apenas o Chance of Admit

[27]:
```
# Após a criação da máscara booleana, podemos colocá-la sobre os dados e␣
↪ocultar os dados que não queremos, os quais são representados por todos os␣
↪valores falsos.
# Faremos isso usando a função .where() no DataFrame.

df.where(mascara_booleana)
```

[27]:
```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1               337.0        118.0                4.0                  4.5
2               324.0        107.0                4.0                  4.0
3               316.0        104.0                3.0                  3.0
4               322.0        110.0                3.0                  3.5
5                 NaN          NaN                NaN                  NaN
…                 …          …                  …                    …
396             324.0        110.0                3.0                  3.5
397             325.0        107.0                3.0                  3.0
398             330.0        116.0                4.0                  5.0
399               NaN          NaN                NaN                  NaN
400             333.0        117.0                4.0                  5.0

            Letter of Recommendation  CGPA  Research  Chance of Admit
Serial No.
1                                4.5  9.65       1.0             0.92
2                                4.5  8.87       1.0             0.76
3                                3.5  8.00       1.0             0.72
4                                2.5  8.67       1.0             0.80
5                                NaN   NaN       NaN              NaN
…                              …   …       …               …
396                              3.5  9.04       1.0             0.82
397                              3.5  9.11       1.0             0.84
398                              4.5  9.45       1.0             0.91
399                              NaN   NaN       NaN              NaN
400                              4.0  9.66       1.0             0.95

[400 rows x 8 columns]
```

Portanto, vemos que o DataFrame resultante mantém os valores do índice original e apenas os dados que atendem à condição foram mostrados, de modo que todas as linhas que não atenderam à condição possuem dados NaN, mas essas linhas NÃO foram descartadas do nosso conjunto de dados.

[28]:
```
# Mas e se não quisermosdados NaN?
# Podemos usar a função dropna()
```

```python
df.where(mascara_booleana).dropna().head()
```

[28]:

```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1               337.0        118.0                4.0                  4.5
2               324.0        107.0                4.0                  4.0
3               316.0        104.0                3.0                  3.0
4               322.0        110.0                3.0                  3.5
6               330.0        115.0                5.0                  4.5

            Letter of Recommendation  CGPA  Research  Chance of Admit
Serial No.
1                                4.5  9.65       1.0             0.92
2                                4.5  8.87       1.0             0.76
3                                3.5  8.00       1.0             0.72
4                                2.5  8.67       1.0             0.80
6                                3.0  9.34       1.0             0.90
```

Assim, os dados que não satisfizeram a condição e ficaram com NaN são descartados, mas perceba que primeiro foi filtrado, os dados que não satisfizeram ficaram com NaN e só depois esses dados foram removidos. Isso perde eficiencia.

Apesar de bastante útil, a função .where() não é muito usada, pois os desenvolvedores do Pandas criaram uma sintaxe abreviada que combina where() e dropna(), representando ambos ao mesmo tempo sem perder a eficiência.

[29]:

```python
# Dá para filtrar de forma direta sem perder a eficiência.
# Basta utilizar a máscara booleana dentro do operador de indexação␣
↪diretamente, como se fosse aplicar em uma matriz.
df[mascara_booleana].head()
```

[29]:

```
            GRE Score  TOEFL Score  University Rating  Statement of Pupose  \
Serial No.
1                 337          118                  4                  4.5
2                 324          107                  4                  4.0
3                 316          104                  3                  3.0
4                 322          110                  3                  3.5
6                 330          115                  5                  4.5

            Letter of Recommendation  CGPA  Research  Chance of Admit
Serial No.
1                                4.5  9.65         1             0.92
2                                4.5  8.87         1             0.76
3                                3.5  8.00         1             0.72
4                                2.5  8.67         1             0.80
6                                3.0  9.34         1             0.90
```

[30]:
```python
# Vamos entender a combinação de várias máscaras booleanas, tais como múltiplos␣
↪critérios para inclusão.
# Isso é feito com "and", caso ambas as máscaras devam ser verdadeiras para␣
↪queum valor verdadeiro esteja na máscara final, ou "or", se apenas uma␣
↪máscara precisa ser verdadeira

# Isso não é muito natural em Pandas, por exemplo, se quisermos pegar duas␣
↪Séries booleanas e usar o "and" junto, fazemos:

mascara_booleana = (df[['Chance of Admit']] > 0.7) and (df[['Chance of Admit']]␣
↪< 0.9)

# Isso vai dar erro
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_23180/2294049538.py in ?()
      2 # Isso é feito com "and", caso ambas as máscaras devam ser verdadeiras␣
↪para queum valor verdadeiro esteja na máscara final, ou "or", se apenas uma␣
↪máscara precisa ser verdadeira
      3
      4 # Isso não é muito natural em Pandas, por exemplo, se quisermos pegar␣
↪duas Séries booleanas e usar o "and" junto, fazemos:
      5
----> 6 mascara_booleana = (df[['Chance of Admit']] > 0.7) and (df[['Chance of␣
↪Admit']] < 0.9)
      7
      8 # Isso vai dar erro

~/.pyenv/versions/3.11.9/lib/python3.11/site-packages/pandas/core/generic.py in ?
↪(self)
   1511     @final
   1512     def __bool__(self) -> NoReturn:
-> 1513         raise ValueError(
   1514             f"The truth value of a {type(self).__name__} is ambiguous.
   1515             "Use a.empty, a.bool(), a.item(), a.any() or a.all()."
   1516         )

ValueError: The truth value of a DataFrame is ambiguous. Use a.empty, a.bool(),␣
↪a.item(), a.any() or a.all().
```

[ ]:
```python
# Esse problema é porque você tem um objeto de Séries, e o Python, por trás,␣
↪não sabe como comparar duas Séries usando "and" ou "or". Em vez disso, o␣
↪autor do Pandas os substituiu por "|" e "&" para lidarmos com isso.

# JEITO CERTO
```

```
mascara_booleana = (df[['Chance of Admit']] > 0.7) & (df[['Chance of Admit']] <␣
↪0.9)
mascara_booleana
```

[ ]:
```
             Chance of Admit
Serial No.
1                      False
2                       True
3                       True
4                       True
5                      False
…                      …
396                     True
397                     True
398                    False
399                    False
400                    False

[400 rows x 1 columns]
```

ATENÇÃO: Um erro comum é fazer comparações booleanas usando o operador "&" mas não colocar o "()" em torno dos termos

[ ]:
```
# Outra forma de fazer isso é simplesmente se livrar completamente do operador␣
↪de comparação e, em vez disso, usar as funções built in que imitam essa␣
↪abordagem
mascara_booleana = df[['Chance of Admit']].gt(0.7) & df[['Chance of Admit']].
↪lt(0.9)
mascara_booleana
```

[ ]:
```
             Chance of Admit
Serial No.
1                      False
2                       True
3                       True
4                       True
5                      False
…                      …
396                     True
397                     True
398                    False
399                    False
400                    False

[400 rows x 1 columns]
```

- .gt() → greater than → maior que (>)

- .lt() → less than → menor que (<)

[ ]:
```
# Podemos fazer de forma encadeada
mascara_booleana = df['Chance of Admit'].gt(0.7).lt(0.9)
mascara_booleana
```

[ ]:
```
Serial No.
1      False
2      False
3      False
4      False
5       True
       …
396    False
397    False
398    False
399     True
400    False
Name: Chance of Admit, Length: 400, dtype: bool
```

22_indexacao_dataframes

February 27, 2026

1 Indexação de DataFrames

Como vimos, tanto Séries como DataFrames podem ter índices aplicados. O índice é basicamente um rótulo de cada linha, e sabemos que linhas correspodem ao eixo (axis) zero. Os índices podem ser gerados automaticamente quando criamos uma nova série sem um índice e, nesse caso, obtemos um valor numérico, ou eles podem ser definitos explicitamente quando usamos o objeto dicionário e criamos uma nova Série ou quando carregamos arquivo CSV e definimos os parâmetros apropriados. Outra opção para configuração de um índice é usar a função set_index(). Essa função usa uma lista de colunas como parâmetro e promove tais colunas como índice.

[32]:
```
# Importanto a biblioteca
import pandas as pd
```

[33]:
```
# A função set_index() é um processo destrutivo, e não mantém o índice atual.␣
↪Se você quiser manter o índice atual, precisa criar manualmente uma nova␣
↪coluna e copiar nela os valores do atributo índice.

df = pd.read_csv('datasets/Admission_Predict.csv', index_col = 0)
df.head()
```

[33]:
```
            GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
Serial No.
1                 337          118                  4  4.5   4.5  9.65
2                 324          107                  4  4.0   4.5  8.87
3                 316          104                  3  3.0   3.5  8.00
4                 322          110                  3  3.5   2.5  8.67
5                 314          103                  2  2.0   3.0  8.21

            Research  Chance of Admit
Serial No.
1                  1             0.92
2                  1             0.76
3                  1             0.72
4                  1             0.80
5                  0             0.65
```

[34]:
```

```

```
# Vamos supor que não queremos mais como índice do DataFrame o número de␣
↪serial, mas, em vez disso, a coluna "Chance of Admit", porém vamos manter a␣
↪coluna "Serial No." para usar depois. Logo, iremos preservar a coluna␣
↪"Serial No." em uma nova coluna.
# Podemos fazer isso usando o operador de indexação com a str que é o nome da␣
↪coluna. Usaremos a função set_index para atribuir como índice a coluna␣
↪"Chance of Admit"

# Copiando os dados da coluna índice para a nova coluna
df['Numero de Série'] = df.index

# Agora podemos definir como o índice uma outra coluna, sem que perdessemos o␣
↪índice inicial
df.set_index('Chance of Admit ', inplace = True)
df.head()
```

[34]:

```
                  GRE Score  TOEFL Score  University Rating  SOP  LOR   CGPA  \
Chance of Admit
0.92                    337          118                  4  4.5   4.5  9.65
0.76                    324          107                  4  4.0   4.5  8.87
0.72                    316          104                  3  3.0   3.5  8.00
0.80                    322          110                  3  3.5   2.5  8.67
0.65                    314          103                  2  2.0   3.0  8.21

                  Research  Numero de Série
Chance of Admit
0.92                     1                1
0.76                     1                2
0.72                     1                3
0.80                     1                4
0.65                     0                5
```

Lembrando que o inplace é para a mudança ocorrer no DataFrame original, uma vez que a função set_index retorna uma cópia, sem alteração do original.

Vemos que criamos um novo índice de uma coluna existente, e o índice tem o mesmo nome da coluna.

[35]:

```
# Podemos resetar o índice usando a função reset_index(). Ela promove o índice,␣
↪tornando-o uma coluna e cria uma coluna índice padrão numerada.
df.reset_index(inplace = True)
df.head()
```

[35]:

```
   Chance of Admit   GRE Score  TOEFL Score  University Rating  SOP  LOR   \
0             0.92        337          118                  4  4.5   4.5
1             0.76        324          107                  4  4.0   4.5
2             0.72        316          104                  3  3.0   3.5
3             0.80        322          110                  3  3.5   2.5
```

```
4              0.65        314          103                   2  2.0   3.0

   CGPA  Research  Numero de Série
0  9.65         1                1
1  8.87         1                2
2  8.00         1                3
3  8.67         1                4
4  8.21         0                5
```

Perceba que "Chance of Admit" é promovida de volta como uma coluna e temos um índice numérico

[36]:
```
# Um bom recurso do Pandas é a indexação em multi-nível. Isso é semelhante ao␣
↪uso de chaves compostas em sistemas de bancos de dados relacionais.
# Para cria um índice multi-nível nós usamos set_index() e informamos a lista␣
↪de colunas que estamos interessados

df = pd.read_csv('datasets/census.csv')
df.head()
```

[36]:
```
   SUMLEV  REGION  DIVISION  STATE  COUNTY   STNAME         CTYNAME  \
0      40       3         6      1       0  Alabama         Alabama
1      50       3         6      1       1  Alabama  Autauga County
2      50       3         6      1       3  Alabama  Baldwin County
3      50       3         6      1       5  Alabama  Barbour County
4      50       3         6      1       7  Alabama     Bibb County

   CENSUS2010POP  ESTIMATESBASE2010  POPESTIMATE2010  …  RDOMESTICMIG2011  \
0        4779736            4780127          4785161  …          0.002295
1          54571              54571            54660  …          7.242091
2         182265             182265           183193  …         14.832960
3          27457              27457            27341  …         -4.728132
4          22915              22919            22861  …         -5.527043

   RDOMESTICMIG2012  RDOMESTICMIG2013  RDOMESTICMIG2014  RDOMESTICMIG2015  \
0         -0.193196          0.381066          0.582002         -0.467369
1         -2.915927         -3.012349          2.265971         -2.530799
2         17.647293         21.845705         19.243287         17.197872
3         -2.500690         -7.056824         -3.904217        -10.543299
4         -5.068871         -6.201001         -0.177537          0.177258

   RNETMIG2011  RNETMIG2012  RNETMIG2013  RNETMIG2014  RNETMIG2015
0     1.030015     0.826644     1.383282     1.724718     0.712594
1     7.606016    -2.626146    -2.722002     2.592270    -2.187333
2    15.844176    18.559627    22.727626    20.317142    18.293499
3    -4.874741    -2.758113    -7.167664    -3.978583   -10.543299
4    -5.088389    -4.363636    -5.403729     0.754533     1.107861
```

```
[5 rows x 100 columns]
```

[37]:
```python
# Por exemplo, nesse conjunto de dados, há dois níveis de sumarização, um que␣
↪contém dados sumarizados (informações brutas condensadas) de todo o país, e␣
↪outro que contém dados sumarizados de cada estado.
# Queremos ver a lista de todos os valores únicos em uma determinada coluna.
# Nesse DataFrame, vemos que os possíveis valores para a coluna "SUMLEV" usando␣
↪a função unique()

df["SUMLEV"].unique()
```

[37]:
```
array([40, 50])
```

Perceba que o resultado nos mostra dois valores apenas, ou seja, a coluna "SUMLEV" só tem dois valores possíveis, ou 40 (Estado) ou 50 (Condado).

[38]:
```python
# Vamos excluir todas as linhas que são resumos no nível estadual e mantenhamos␣
↪dados de condados.
df = df[df['SUMLEV'] == 50]
df.head()
```

[38]:
```
   SUMLEV  REGION  DIVISION  STATE  COUNTY   STNAME         CTYNAME  \
1      50       3         6      1       1  Alabama  Autauga County
2      50       3         6      1       3  Alabama  Baldwin County
3      50       3         6      1       5  Alabama  Barbour County
4      50       3         6      1       7  Alabama     Bibb County
5      50       3         6      1       9  Alabama   Blount County

   CENSUS2010POP  ESTIMATESBASE2010  POPESTIMATE2010  …  RDOMESTICMIG2011  \
1          54571              54571            54660  …          7.242091
2         182265             182265           183193  …         14.832960
3          27457              27457            27341  …         -4.728132
4          22915              22919            22861  …         -5.527043
5          57322              57322            57373  …          1.807375

   RDOMESTICMIG2012  RDOMESTICMIG2013  RDOMESTICMIG2014  RDOMESTICMIG2015  \
1         -2.915927         -3.012349          2.265971         -2.530799
2         17.647293         21.845705         19.243287         17.197872
3         -2.500690         -7.056824         -3.904217        -10.543299
4         -5.068871         -6.201001         -0.177537          0.177258
5         -1.177622         -1.748766         -2.062535         -1.369970

   RNETMIG2011  RNETMIG2012  RNETMIG2013  RNETMIG2014  RNETMIG2015
1     7.606016    -2.626146    -2.722002     2.592270    -2.187333
2    15.844176    18.559627    22.727626    20.317142    18.293499
3    -4.874741    -2.758113    -7.167664    -3.978583   -10.543299
4    -5.088389    -4.363636    -5.403729     0.754533     1.107861
```

```
5     1.859511    -0.848580    -1.402476    -1.577232    -0.884411

[5 rows x 100 columns]
```

[39]:
```
# Além disso, enquanto esse conjunto de dados for interessante por um número de␣
↪diferentes razões, vamos reduzir nosso conjunto de dados para ver apenas o␣
↪total de estimativas populacionais e o número total de nascimentos.

# Podíamos excluir todas as colunas que não queremos, porém, neste caso, como␣
↪existem muitas, é mais fácil escrever as que queremos manter.
# Podemos começar criando uma lista de nomes das colunas que queremos manter
colunas_matidas =␣
↪['STNAME','CTYNAME','BIRTHS2010','BIRTHS2011','BIRTHS2012','BIRTHS2013',
                  ␣
↪'BIRTHS2014','BIRTHS2015','POPESTIMATE2010','POPESTIMATE2011',
                  ␣
↪'POPESTIMATE2012','POPESTIMATE2013','POPESTIMATE2014','POPESTIMATE2015']

df = df[colunas_matidas]
df.head()
```

[39]:
```
    STNAME         CTYNAME  BIRTHS2010  BIRTHS2011  BIRTHS2012  BIRTHS2013  \
1  Alabama  Autauga County         151         636         615         574
2  Alabama  Baldwin County         517        2187        2092        2160
3  Alabama  Barbour County          70         335         300         283
4  Alabama     Bibb County          44         266         245         259
5  Alabama   Blount County         183         744         710         646

   BIRTHS2014  BIRTHS2015  POPESTIMATE2010  POPESTIMATE2011  POPESTIMATE2012  \
1         623         600            54660            55253            55175
2        2186        2240           183193           186659           190396
3         260         269            27341            27226            27159
4         247         253            22861            22733            22642
5         618         603            57373            57711            57776

   POPESTIMATE2013  POPESTIMATE2014  POPESTIMATE2015
1            55038            55290            55347
2           195126           199713           203709
3            26973            26815            26489
4            22512            22549            22583
5            57734            57658            57673
```

[40]:
```
# Os dados do Censo dos EUA possibilitam separar as estimativas populacionais␣
↪por estado e condado
# Podemos carregar os dados e usando set_index() para combinar valores de␣
↪estados e de condados.
```

```python
df.set_index(['STNAME', 'CTYNAME'], inplace = True)
df.head(10)
```

[40]:

```
                          BIRTHS2010  BIRTHS2011  BIRTHS2012  BIRTHS2013  \
STNAME  CTYNAME
Alabama Autauga County          151         636         615         574
        Baldwin County          517        2187        2092        2160
        Barbour County           70         335         300         283
        Bibb County              44         266         245         259
        Blount County           183         744         710         646
        Bullock County           39         169         122         132
        Butler County            65         276         241         240
        Calhoun County          317        1382        1357        1309
        Chambers County          81         401         393         404
        Cherokee County          55         221         245         232

                          BIRTHS2014  BIRTHS2015  POPESTIMATE2010  \
STNAME  CTYNAME
Alabama Autauga County          623         600            54660
        Baldwin County         2186        2240           183193
        Barbour County          260         269            27341
        Bibb County             247         253            22861
        Blount County           618         603            57373
        Bullock County          118         123            10887
        Butler County           267         257            20944
        Calhoun County         1355        1335           118437
        Chambers County         421         429            34098
        Cherokee County         261         250            25976

                          POPESTIMATE2011  POPESTIMATE2012  POPESTIMATE2013  \
STNAME  CTYNAME
Alabama Autauga County             55253            55175            55038
        Baldwin County            186659           190396           195126
        Barbour County             27226            27159            26973
        Bibb County                22733            22642            22512
        Blount County              57711            57776            57734
        Bullock County             10629            10606            10628
        Butler County              20673            20408            20261
        Calhoun County            117768           117286           116575
        Chambers County            33993            34075            34153
        Cherokee County            26080            26023            26084

                          POPESTIMATE2014  POPESTIMATE2015
STNAME  CTYNAME
Alabama Autauga County             55290            55347
        Baldwin County            199713           203709
```

```
Barbour County          26815          26489
Bibb County             22549          22583
Blount County           57658          57673
Bullock County          10829          10696
Butler County           20276          20154
Calhoun County         115993         115620
Chambers County         34052          34123
Cherokee County         25995          25859
```

[ ]:
```
# Como podemos consultar esse DataFrame?
# Por exemplo, vimos que o atributo "loc" do DataFrame pode ter vários␣
↪argumentos, e ele poderia consultar ambas linhas e colunas.
# Quando usamos multi-index precisamos fornecer os argumentos na ordem do nível␣
↪que desejamos consultar. Dentro do índice, cada coluna é chamada de 'nível',␣
↪de modo que a coluna mais externa é o nível zero.

# Então, se quisermos ver os resultados da população do condado de Washtenaw,␣
↪no estado de Michigan, o primeiro argumento seria "Michigan" e o segundo␣
↪"Washtenaw County"
df.loc['Michigan', 'Washtenaw County']
```

[ ]:
```
BIRTHS2010            977
BIRTHS2011           3826
BIRTHS2012           3780
BIRTHS2013           3662
BIRTHS2014           3683
BIRTHS2015           3709
POPESTIMATE2010    345563
POPESTIMATE2011    349048
POPESTIMATE2012    351213
POPESTIMATE2013    354289
POPESTIMATE2014    357029
POPESTIMATE2015    358880
Name: (Michigan, Washtenaw County), dtype: int64
```

[ ]:
```
# E se quisermos filtrar apenas duas colunas?
# Nesse caso, precisamos NECESSARIAMENTE passar os níveis entre TUPLAS, para␣
↪conseguirmos separar as colunas
df.loc[('Michigan', 'Washtenaw County'), ['BIRTHS2010', 'BIRTHS2011',␣
↪'BIRTHS2012']]
```

[ ]:
```
BIRTHS2010     977
BIRTHS2011    3826
BIRTHS2012    3780
Name: (Michigan, Washtenaw County), dtype: int64
```

[50]:
```
# E se quisermos filtrar mais níveis?
# Então devemos passar uma lista de tuplas
df.loc[[('Michigan', 'Washtenaw County'), ('Alabama', 'Autauga County')],␣
↪['BIRTHS2010', 'BIRTHS2011', 'BIRTHS2012']]
```

[50]:

		BIRTHS2010	BIRTHS2011	BIRTHS2012
STNAME	CTYNAME			
Michigan	Washtenaw County	977	3826	3780
Alabama	Autauga County	151	636	615

[ ]:
```
# Se for o mesmo nível 0 mas diferentes níveis 1, podemos utilizar a tupla, com␣
↪uma lista dentro da tupla
df.loc[('Alabama', ['Autauga County', 'Baldwin County']), ['BIRTHS2010',␣
↪'BIRTHS2011', 'BIRTHS2012']]



# Isso evita ter que ficar repetindo Alabama, por exemplo, df.loc[[('Alabama',␣
↪'Autauga County'), ('Alabama', 'Baldwin County')], ['BIRTHS2010',␣
↪'BIRTHS2011']]
```

[ ]:

		BIRTHS2010	BIRTHS2011
STNAME	CTYNAME		
Alabama	Autauga County	151	636
	Baldwin County	517	2187

Acessar um índice composto específico
df.loc[('nivel0', 'nivel1')]

Acessar um índice composto + colunas específicas
df.loc[('nivel0', 'nivel1'), ['COL1', 'COL2']]

Acessar todos os valores de um nível 0
df.loc['nivel0']

Mesmo nível 0, vários nível 1
df.loc[('nivel0', ['nivel1a', 'nivel1b'])]

Mesmo nível 0, vários nível 1 + colunas específicas
df.loc[('nivel0', ['nivel1a', 'nivel1b']), ['COL1', 'COL2']]

Vários pares específicos
df.loc[[('nivel0a', 'nivel1a'), ('nivel0b', 'nivel1b')]]

23_valores_ausentes

February 27, 2026

1 Valores ausentes

Vimos uma prévia de como o Pandas lida com valores ausentes usando o tipo NaN. Valores ausentes são muito comuns em atividades de limpeza de dados, e valores ausentes podem existir por vários motivos:

- Pode haver, dentro de uma pesquisa, respostas que o respondente não forneceu, ou seja, uma OMISSÃO. Esse dado ausente, chamamos de **Missing at Random** (ausente aleatoriamente) se houvver outras variáveis que possam ser usadas para prever a variável que está ausente. Se os dados faltantes não possuem relação com nenhuma outra variável chamamos esses dados de **Missing Completely at Random** (desaparecidos completamente de forma aleatória)
- Dados ausentes por não terem sido coletados

[41]:
```
# Importando a biblioteca
import pandas as pd
```

[42]:
```
# Pandas é muito bom em detectar valores ausentes diretaemnte de arquivos como␣
↪CSV.
# Embora a maioria dos valores ausentes geralmente seja formatada como NaN,␣
↪null, None, NA, às vezes os valores ausentes não são rotulados de forma tão␣
↪clara, por exemplo, utilizar 99 em categorias binárias para indicar que é um␣
↪valor ausente.

# A função pandas read_csv() tem um parâmetro chamado "na_values" que nos␣
↪permite especificar o formato dos valores ausentes. Ele permite que␣
↪sequências escalares, listas ou dicionários sejam usados.
df = pd.read_csv('datasets/class_grades.csv')
df.head(10)
```

[42]:

	Prefix	Assignment	Tutorial	Midterm	TakeHome	Final
0	5	57.14	34.09	64.38	51.48	52.50
1	8	95.05	105.49	67.50	99.07	68.33
2	8	83.70	83.17	NaN	63.15	48.89
3	7	NaN	NaN	49.38	105.93	80.56
4	8	91.32	93.64	95.00	107.41	73.89
5	7	95.00	92.58	93.12	97.78	68.06
6	8	95.05	102.99	56.25	99.07	50.00
7	7	72.85	86.85	60.00	NaN	56.11

```
8        8        84.26      93.10     47.50      18.52  50.83
9        7        90.10      97.55     51.25      88.89  63.61
```

Perceba que existem vários valores ausentes apenas nessas 10 linhas.

[43]:
```python
# Podemos usar a função .isnull() para criar uma máscara booleana de todo o␣
↪quadro de dados. Isso vai funcionar por broadcast
mascara = df.isnull()
mascara
```

[43]:
```
    Prefix  Assignment  Tutorial  Midterm  TakeHome  Final
0    False       False     False    False     False  False
1    False       False     False    False     False  False
2    False       False     False     True     False  False
3    False        True      True    False     False  False
4    False       False     False    False     False  False
..     …           …         …        …         …      …
94   False        True     False    False     False  False
95   False        True     False    False     False  False
96   False       False     False    False     False  False
97   False       False     False    False     False  False
98   False       False     False    False     False  False

[99 rows x 6 columns]
```

[44]:
```python
# Isso pode ser útil para processar linhas com base em determinadas colunas de␣
↪dados. Outra operação útil é poder eliminar todas as linhas que têm dados␣
↪ausentes, o que pode ser feito com a função dropna()
df.dropna().head(10)
```

[44]:
```
    Prefix  Assignment  Tutorial  Midterm  TakeHome  Final
0        5       57.14     34.09    64.38     51.48  52.50
1        8       95.05    105.49    67.50     99.07  68.33
4        8       91.32     93.64    95.00    107.41  73.89
5        7       95.00     92.58    93.12     97.78  68.06
6        8       95.05    102.99    56.25     99.07  50.00
8        8       84.26     93.10    47.50     18.52  50.83
9        7       90.10     97.55    51.25     88.89  63.61
10       7       80.44     90.20    75.00     91.48  39.72
12       8       97.16    103.71    72.50     93.52  63.33
13       7       91.28     83.53    81.25     99.81  92.22
```

Perceba que algumas linhas foram removidas

[45]:
```python
# Uma das funções úteis que o Pandas tem para trabalhar com valores ausentes é␣
↪a função de preenchimento chamada fillna(). Essa função usa vários␣
↪parâmetros, de modo que podemos passar um único valor, chamado de valor␣
↪escalar, para transformar todos os dados ausentes em um único valor
```

```
# Se quiséssemos preencher todos os valores ausentes com zero, podemos usar␣
↪fillna()
df.fillna(0, inplace = True)
```

[45]:

	Prefix	Assignment	Tutorial	Midterm	TakeHome	Final
0	5	57.14	34.09	64.38	51.48	52.50
1	8	95.05	105.49	67.50	99.07	68.33
2	8	83.70	83.17	0.00	63.15	48.89
3	7	0.00	0.00	49.38	105.93	80.56
4	8	91.32	93.64	95.00	107.41	73.89
..	…	…	…	…	…	…
94	8	0.00	103.71	45.00	93.52	61.94
95	7	0.00	80.54	41.25	93.70	39.72
96	8	89.94	102.77	87.50	90.74	87.78
97	7	95.60	76.13	66.25	99.81	85.56
98	8	63.40	97.37	73.12	72.78	77.22

[99 rows x 6 columns]

[ ]:

```
# Também podemos usar o parâmetro na_filter para desativar a filtragem de␣
↪espaço em branco se o espaço em branco for um valor real de interesse.
# OBS: Na prática isso é muito raro, pois em dados sem nenhum NAs, passar␣
↪na_filter = False pode melhorar o desempenho da leitura de um arquivo grande
# pd.read_csv("arquivo.csv", na_filter = False)
"""
Quando pedimos isso, estamos falando: Não procure valores ausentes. Leia tudo␣
↪exatamente como está."

Ou seja:
Ele não converte strings vazias em NaN
Ele não converte "NA" em NaN
Ele simplesmente carrega os dados como texto bruto
"""

df = pd.read_csv('datasets/log.csv')
df.head(20)
```

[ ]:

	time	user	video	playback position	paused	volume
0	1469974424	cheryl	intro.html	5	False	10.0
1	1469974454	cheryl	intro.html	6	NaN	NaN
2	1469974544	cheryl	intro.html	9	NaN	NaN
3	1469974574	cheryl	intro.html	10	NaN	NaN
4	1469977514	bob	intro.html	1	NaN	NaN
5	1469977544	bob	intro.html	1	NaN	NaN
6	1469977574	bob	intro.html	1	NaN	NaN
7	1469977604	bob	intro.html	1	NaN	NaN

```
8   1469974604  cheryl     intro.html                 11    NaN     NaN
9   1469974694  cheryl     intro.html                 14    NaN     NaN
10  1469974724  cheryl     intro.html                 15    NaN     NaN
11  1469974454     sue  advanced.html                 24    NaN     NaN
12  1469974524     sue  advanced.html                 25    NaN     NaN
13  1469974424     sue  advanced.html                 23  False    10.0
14  1469974554     sue  advanced.html                 26    NaN     NaN
15  1469974624     sue  advanced.html                 27    NaN     NaN
16  1469974654     sue  advanced.html                 28    NaN     5.0
17  1469974724     sue  advanced.html                 29    NaN     NaN
18  1469974484  cheryl     intro.html                  7    NaN     NaN
19  1469974514  cheryl     intro.html                  8    NaN     NaN
```

[47]:
```python
# Vamos setar o index com base na coluna 'time'
df = df.set_index('time')

# Vamos ordenar com base no index, pois ao mudar o index, vai ficar desordenado
df = df.sort_index()
df.head(20)
```

[47]:
```
              user          video  playback position paused  volume
time
1469974424  cheryl     intro.html                  5  False    10.0
1469974424     sue  advanced.html                 23  False    10.0
1469974454     sue  advanced.html                 24    NaN     NaN
1469974454  cheryl     intro.html                  6    NaN     NaN
1469974484  cheryl     intro.html                  7    NaN     NaN
1469974514  cheryl     intro.html                  8    NaN     NaN
1469974524     sue  advanced.html                 25    NaN     NaN
1469974544  cheryl     intro.html                  9    NaN     NaN
1469974554     sue  advanced.html                 26    NaN     NaN
1469974574  cheryl     intro.html                 10    NaN     NaN
1469974604  cheryl     intro.html                 11    NaN     NaN
1469974624     sue  advanced.html                 27    NaN     NaN
1469974634  cheryl     intro.html                 12    NaN     NaN
1469974654     sue  advanced.html                 28    NaN     5.0
1469974664  cheryl     intro.html                 13    NaN     NaN
1469974694  cheryl     intro.html                 14    NaN     NaN
1469974724  cheryl     intro.html                 15    NaN     NaN
1469974724     sue  advanced.html                 29    NaN     NaN
1469974754     sue  advanced.html                 30    NaN     NaN
1469974824     sue  advanced.html                 31    NaN     NaN
```

[54]:
```python
# Se observarmos atentamente a saída, perceberemos que o índice não é realmente␣
↪único. Parece que dois usuários conseguem usar o sistema ao mesmo tempo.
# Nesse caso, vamos redefinir o índice e usar uma indexação multi-nível sobre␣
↪pontualidade e o usuário
```

```
df.reset_index(inplace = True)
df.set_index(['time', 'user'], inplace = True)
df.sort_index(inplace = True)
df
```

[54]:

		video	playback position	paused	volume
time	user				
1469974424	cheryl	intro.html	5	False	10.0
	sue	advanced.html	23	False	10.0
1469974454	cheryl	intro.html	6	NaN	NaN
	sue	advanced.html	24	NaN	NaN
1469974484	cheryl	intro.html	7	NaN	NaN
1469974514	cheryl	intro.html	8	NaN	NaN
1469974524	sue	advanced.html	25	NaN	NaN
1469974544	cheryl	intro.html	9	NaN	NaN
1469974554	sue	advanced.html	26	NaN	NaN
1469974574	cheryl	intro.html	10	NaN	NaN
1469974604	cheryl	intro.html	11	NaN	NaN
1469974624	sue	advanced.html	27	NaN	NaN
1469974634	cheryl	intro.html	12	NaN	NaN
1469974654	sue	advanced.html	28	NaN	5.0
1469974664	cheryl	intro.html	13	NaN	NaN
1469974694	cheryl	intro.html	14	NaN	NaN
1469974724	cheryl	intro.html	15	NaN	NaN
	sue	advanced.html	29	NaN	NaN
1469974754	sue	advanced.html	30	NaN	NaN
1469974824	sue	advanced.html	31	NaN	NaN
1469974854	sue	advanced.html	32	NaN	NaN
1469974924	sue	advanced.html	33	NaN	NaN
1469977424	bob	intro.html	1	True	10.0
1469977454	bob	intro.html	1	NaN	NaN
1469977484	bob	intro.html	1	NaN	NaN
1469977514	bob	intro.html	1	NaN	NaN
1469977544	bob	intro.html	1	NaN	NaN
1469977574	bob	intro.html	1	NaN	NaN
1469977604	bob	intro.html	1	NaN	NaN
1469977634	bob	intro.html	1	NaN	NaN
1469977664	bob	intro.html	1	NaN	NaN
1469977694	bob	intro.html	1	NaN	NaN
1469977724	bob	intro.html	1	NaN	NaN

[56]:

```
# Agora que temos os dados indexados e classificados adequadamente, podemos␣
↪preencher os dados ausentes usando ffill.
# É bom lembrar que, ao lidar com valores ausentes, você pode lidar com colunas␣
↪indivudais ou conjuntos de colunass projetando-os para que não precise␣
↪corrigir todos os valores ausentes em um comando.
```

```
df = df.ffill()
df.head()
```

[56]:

```
                          video  playback position paused  volume
time       user
1469974424 cheryl     intro.html                  5  False    10.0
           sue     advanced.html                 23  False    10.0
1469974454 cheryl     intro.html                  6  False    10.0
           sue     advanced.html                 24  False    10.0
1469974484 cheryl     intro.html                  7  False    10.0
```

Essa função ffill() faz:

- Ele preenche valores ausentes com o último valor válido acima.

Já a fillna() preenche com um valor especificado.

[58]:

```
# Também podemos fazer um preenchimento personalizado para substituir os␣
↪valores com a função replace(). Ela permite a substituição de várias␣
↪abordagens, valor para valor, lista, dicionário, regex, então vamos gerar um␣
↪exemplo simples.
df = pd.DataFrame({'A': [1, 1, 2, 3, 4],
                   'B': [3, 6, 3, 8, 9],
                   'C': ['a', 'b', 'c', 'd', 'e']})
df
```

[58]:

	A	B	C
0	1	3	a
1	1	6	b
2	2	3	c
3	3	8	d
4	4	9	e

[61]:

```
# Poderíamos substituir aquele por 100. Então vamos tentar a abordagem de valor␣
↪para valor
# Vamos passar na função replace() primeiro o valor que vamos substituir e␣
↪depois o valor que entrará no lugar.
df.replace(1, 100, inplace = True)
```

[61]:

	A	B	C
0	100	3	a
1	100	6	b
2	2	3	c
3	3	8	d
4	4	9	e

[62]:
```python
# Agora vamos mudar tanto o 100 quanto o 3 para 103 e 300
df.replace([100, 3], [103, 300])
```

[62]:

	A	B	C
0	103	300	a
1	103	6	b
2	2	300	c
3	300	8	d
4	4	9	e

[64]:
```python
# A substituição do Pandas também suporta regex
df = pd.read_csv('datasets/log.csv')
df
```

[64]:

	time	user	video	playback position	paused	volume
0	1469974424	cheryl	intro.html	5	False	10.0
1	1469974454	cheryl	intro.html	6	NaN	NaN
2	1469974544	cheryl	intro.html	9	NaN	NaN
3	1469974574	cheryl	intro.html	10	NaN	NaN
4	1469977514	bob	intro.html	1	NaN	NaN
5	1469977544	bob	intro.html	1	NaN	NaN
6	1469977574	bob	intro.html	1	NaN	NaN
7	1469977604	bob	intro.html	1	NaN	NaN
8	1469974604	cheryl	intro.html	11	NaN	NaN
9	1469974694	cheryl	intro.html	14	NaN	NaN
10	1469974724	cheryl	intro.html	15	NaN	NaN
11	1469974454	sue	advanced.html	24	NaN	NaN
12	1469974524	sue	advanced.html	25	NaN	NaN
13	1469974424	sue	advanced.html	23	False	10.0
14	1469974554	sue	advanced.html	26	NaN	NaN
15	1469974624	sue	advanced.html	27	NaN	NaN
16	1469974654	sue	advanced.html	28	NaN	5.0
17	1469974724	sue	advanced.html	29	NaN	NaN
18	1469974484	cheryl	intro.html	7	NaN	NaN
19	1469974514	cheryl	intro.html	8	NaN	NaN
20	1469974754	sue	advanced.html	30	NaN	NaN
21	1469974824	sue	advanced.html	31	NaN	NaN
22	1469974854	sue	advanced.html	32	NaN	NaN
23	1469974924	sue	advanced.html	33	NaN	NaN
24	1469977424	bob	intro.html	1	True	10.0
25	1469977454	bob	intro.html	1	NaN	NaN
26	1469977484	bob	intro.html	1	NaN	NaN
27	1469977634	bob	intro.html	1	NaN	NaN
28	1469977664	bob	intro.html	1	NaN	NaN
29	1469974634	cheryl	intro.html	12	NaN	NaN
30	1469974664	cheryl	intro.html	13	NaN	NaN
31	1469977694	bob	intro.html	1	NaN	NaN

```
32  1469977724     bob     intro.html                  1    NaN     NaN
```

[65]:
```python
# Para substituir usando regex, criamos o primeiro parâmetro para substituir o␣
↪padrão regex que queremos substituir, o segundo parâmetro, o valor que␣
↪queremos emitir em uma correspondência, e então passamos um terceiro␣
↪parâmetro que diz apenas que regex é igual a true.

df.replace(to_replace = '.*.html$', value = 'webpage', regex = True)
```

[65]:

	time	user	video	playback position	paused	volume
0	1469974424	cheryl	webpage	5	False	10.0
1	1469974454	cheryl	webpage	6	NaN	NaN
2	1469974544	cheryl	webpage	9	NaN	NaN
3	1469974574	cheryl	webpage	10	NaN	NaN
4	1469977514	bob	webpage	1	NaN	NaN
5	1469977544	bob	webpage	1	NaN	NaN
6	1469977574	bob	webpage	1	NaN	NaN
7	1469977604	bob	webpage	1	NaN	NaN
8	1469974604	cheryl	webpage	11	NaN	NaN
9	1469974694	cheryl	webpage	14	NaN	NaN
10	1469974724	cheryl	webpage	15	NaN	NaN
11	1469974454	sue	webpage	24	NaN	NaN
12	1469974524	sue	webpage	25	NaN	NaN
13	1469974424	sue	webpage	23	False	10.0
14	1469974554	sue	webpage	26	NaN	NaN
15	1469974624	sue	webpage	27	NaN	NaN
16	1469974654	sue	webpage	28	NaN	5.0
17	1469974724	sue	webpage	29	NaN	NaN
18	1469974484	cheryl	webpage	7	NaN	NaN
19	1469974514	cheryl	webpage	8	NaN	NaN
20	1469974754	sue	webpage	30	NaN	NaN
21	1469974824	sue	webpage	31	NaN	NaN
22	1469974854	sue	webpage	32	NaN	NaN
23	1469974924	sue	webpage	33	NaN	NaN
24	1469977424	bob	webpage	1	True	10.0
25	1469977454	bob	webpage	1	NaN	NaN
26	1469977484	bob	webpage	1	NaN	NaN
27	1469977634	bob	webpage	1	NaN	NaN
28	1469977664	bob	webpage	1	NaN	NaN
29	1469974634	cheryl	webpage	12	NaN	NaN
30	1469974664	cheryl	webpage	13	NaN	NaN
31	1469977694	bob	webpage	1	NaN	NaN
32	1469977724	bob	webpage	1	NaN	NaN

24_manipulacao_dataframe

February 27, 2026

1 Manipulação de DataFrame

[2]:
```
# Importando a biblioteca
import pandas as pd

# Lendo o arquivo
df = pd.read_csv('datasets/presidents.csv')

df.head()
```

[2]:
```
   #          President             Born      Age atstart of presidency  \
0  1  George Washington  Feb 22, 1732[a]  57 years, 67 daysApr 30, 1789
1  2         John Adams  Oct 30, 1735[a]  61 years, 125 daysMar 4, 1797
2  3   Thomas Jefferson  Apr 13, 1743[a]  57 years, 325 daysMar 4, 1801
3  4      James Madison  Mar 16, 1751[a]  57 years, 353 daysMar 4, 1809
4  5       James Monroe     Apr 28, 1758  58 years, 310 daysMar 4, 1817

         Age atend of presidency Post-presidencytimespan          Died  \
0   65 years, 10 daysMar 4, 1797       2 years, 285 days  Dec 14, 1799
1  65 years, 125 daysMar 4, 1801      25 years, 122 days   Jul 4, 1826
2  65 years, 325 daysMar 4, 1809      17 years, 122 days   Jul 4, 1826
3  65 years, 353 daysMar 4, 1817      19 years, 116 days  Jun 28, 1836
4  66 years, 310 daysMar 4, 1825       6 years, 122 days   Jul 4, 1831

                  Age
0  67 years, 295 days
1  90 years, 247 days
2   83 years, 82 days
3  85 years, 104 days
4   73 years, 67 days
```

[3]:
```
# Vamos colocar o index para a coluna '#'
df.set_index('#', inplace = True)
df.head()
```

[3]:
```
           President             Born      Age atstart of presidency  \
#
```

```
1  George Washington  Feb 22, 1732[a]  57 years, 67 daysApr 30, 1789
2         John Adams  Oct 30, 1735[a]  61 years, 125 daysMar 4, 1797
3   Thomas Jefferson  Apr 13, 1743[a]  57 years, 325 daysMar 4, 1801
4      James Madison  Mar 16, 1751[a]  57 years, 353 daysMar 4, 1809
5       James Monroe     Apr 28, 1758  58 years, 310 daysMar 4, 1817

         Age atend of presidency Post-presidencytimespan          Died  \
#
1   65 years, 10 daysMar 4, 1797       2 years, 285 days  Dec 14, 1799
2  65 years, 125 daysMar 4, 1801      25 years, 122 days   Jul 4, 1826
3  65 years, 325 daysMar 4, 1809      17 years, 122 days   Jul 4, 1826
4  65 years, 353 daysMar 4, 1817      19 years, 116 days  Jun 28, 1836
5  66 years, 310 daysMar 4, 1825       6 years, 122 days   Jul 4, 1831

                  Age
#
1  67 years, 295 days
2  90 years, 247 days
3   83 years, 82 days
4  85 years, 104 days
5   73 years, 67 days
```

Temos agumas notas de rodapé na coluna 'born', então vamos tirar essas notas, pois elas podem causar problemas. Além disso, também vamos ajustar os presidentes em primeiro nome e sobrenome.

[4]:
```python
# 1º Forma:
# Vamos copiar a coluna de president
df['Primeiro nome'] = df['President']

# Agoram podemos modificar a nova coluna com replace()
df['Primeiro nome'] = df['Primeiro nome'].replace('\s[\w]*', '', regex = True)
df.head()
```

[4]:
```
           President             Born      Age atstart of presidency  \
#
1  George Washington  Feb 22, 1732[a]  57 years, 67 daysApr 30, 1789
2         John Adams  Oct 30, 1735[a]  61 years, 125 daysMar 4, 1797
3   Thomas Jefferson  Apr 13, 1743[a]  57 years, 325 daysMar 4, 1801
4      James Madison  Mar 16, 1751[a]  57 years, 353 daysMar 4, 1809
5       James Monroe     Apr 28, 1758  58 years, 310 daysMar 4, 1817

         Age atend of presidency Post-presidencytimespan          Died  \
#
1   65 years, 10 daysMar 4, 1797       2 years, 285 days  Dec 14, 1799
2  65 years, 125 daysMar 4, 1801      25 years, 122 days   Jul 4, 1826
3  65 years, 325 daysMar 4, 1809      17 years, 122 days   Jul 4, 1826
```

```
4  65 years, 353 daysMar 4, 1817      19 years, 116 days  Jun 28, 1836
5  66 years, 310 daysMar 4, 1825       6 years, 122 days   Jul 4, 1831

                  Age Primeiro nome
#
1  67 years, 295 days        George
2  90 years, 247 days          John
3   83 years, 82 days        Thomas
4  85 years, 104 days         James
5   73 years, 67 days         James
```

Isso que fizemos funciona, porém é meio inadequado e é lento, pois tivemos que fazer uma cópia completa da coluna antes e depois atualizar as strings.

[5]:
```python
# 2º Forma:
# Forma mais genérica, que é a função apply()

# Removendo a coluna que criamos
del df['Primeiro nome']

# A função apply() em um DataFrame pega alguma função arbitrária que você␣
↪codificou e a aplica a uma série, a uma única coluna ou ao DataFrame total,␣
↪de forma iterativa
def separa_nome(linha: pd.Series):
    # Cada chamada dessa função recebe uma linha do DataFrame, a qual é passada␣
↪como uma Série
    # Vamos extrair o primeiro nome e criar uma nova entrada para a série
    linha['Primeiro nome'] = str(linha['President']).split()[0]
    linha['Sobrenome'] = str(linha['President']).split()[-1]

    return linha

# Agora aplicamos isso ao DataFrame indicando que queremos aplicar nas colunas
df = df.apply(separa_nome, axis = 1)
df.head(10)
```

[5]:
```
              President             Born      Age atstart of presidency  \
#
1     George Washington  Feb 22, 1732[a]  57 years, 67 daysApr 30, 1789
2            John Adams  Oct 30, 1735[a]  61 years, 125 daysMar 4, 1797
3      Thomas Jefferson  Apr 13, 1743[a]  57 years, 325 daysMar 4, 1801
4         James Madison  Mar 16, 1751[a]  57 years, 353 daysMar 4, 1809
5          James Monroe     Apr 28, 1758  58 years, 310 daysMar 4, 1817
6     John Quincy Adams     Jul 11, 1767  57 years, 236 daysMar 4, 1825
7        Andrew Jackson     Mar 15, 1767  61 years, 354 daysMar 4, 1829
8      Martin Van Buren      Dec 5, 1782   54 years, 89 daysMar 4, 1837
9   William H. Harrison      Feb 9, 1773   68 years, 23 daysMar 4, 1841
```

```
10           John Tyler     Mar 29, 1790    51 years, 6 daysApr 4, 1841

             Age atend of presidency Post-presidencytimespan          Died  \
#
1       65 years, 10 daysMar 4, 1797       2 years, 285 days  Dec 14, 1799
2      65 years, 125 daysMar 4, 1801      25 years, 122 days   Jul 4, 1826
3      65 years, 325 daysMar 4, 1809      17 years, 122 days   Jul 4, 1826
4      65 years, 353 daysMar 4, 1817      19 years, 116 days  Jun 28, 1836
5      66 years, 310 daysMar 4, 1825       6 years, 122 days   Jul 4, 1831
6      61 years, 236 daysMar 4, 1829      18 years, 356 days  Feb 23, 1848
7      69 years, 354 daysMar 4, 1837        8 years, 96 days   Jun 8, 1845
8       58 years, 89 daysMar 4, 1841      21 years, 142 days  Jul 24, 1862
9   68 years, 54 days Apr 4, 1841[b]                     NaN   Apr 4, 1841
10     54 years, 340 daysMar 4, 1845      16 years, 320 days  Jan 18, 1862

                   Age Primeiro nome   Sobrenome
#
1   67 years, 295 days        George  Washington
2   90 years, 247 days          John       Adams
3    83 years, 82 days        Thomas   Jefferson
4   85 years, 104 days         James     Madison
5    73 years, 67 days         James      Monroe
6   80 years, 227 days          John       Adams
7    78 years, 85 days        Andrew     Jackson
8   79 years, 231 days        Martin       Buren
9    68 years, 54 days       William    Harrison
10  71 years, 295 days          John       Tyler
```

[6]:
```python
# 3º Forma:

# Removendo as colunas que criamos
df.drop(['Primeiro nome', 'Sobrenome'], axis = 1, inplace = True)

# Vamos usar a função extract(), pois ela pega expressões regulares (regex)␣
↪como entrada, atribuindo o valor desejado de grupos de captura que␣
↪correspondem às colunas de saída que estamos interessados
nomes = df['President'].str.extract('(?P<Primeiro_nome>^[\w]*)(?:.*\s)(?
↪P<Sobrenome>[\w]*$)')
nomes
```

[6]:
```
   Primeiro_nome   Sobrenome
#
1         George  Washington
2           John       Adams
3         Thomas   Jefferson
4          James     Madison
5          James      Monroe
```

```
6          John       Adams
7        Andrew     Jackson
8        Martin       Buren
9       William    Harrison
10         John       Tyler
11        James        Polk
12      Zachary      Taylor
13      Millard    Fillmore
14     Franklin      Pierce
15        James    Buchanan
16      Abraham     Lincoln
17       Andrew     Johnson
18     Ulysses       Grant
19   Rutherford       Hayes
20        James    Garfield
21      Chester      Arthur
22       Grover   Cleveland
23     Benjamin    Harrison
24       Grover   Cleveland
25      William    McKinley
26     Theodore   Roosevelt
27      William        Taft
28      Woodrow      Wilson
29       Warren     Harding
30       Calvin    Coolidge
31      Herbert      Hoover
32     Franklin   Roosevelt
33        Harry      Truman
34       Dwight  Eisenhower
35         John     Kennedy
36       Lyndon     Johnson
37      Richard       Nixon
38       Gerald        Ford
39        Jimmy      Carter
40       Ronald      Reagan
41       George        Bush
42         Bill     Clinton
43       George        Bush
44       Barack       Obama
```

OBS: Em regex, o nome do grupo não pode ter espaços

[7]:
```python
# Agora, basta passar isso para o DataFrame
df['Primeiro nome'] = nomes['Primeiro_nome']
df['Sobrenome'] = nomes['Sobrenome']
df.head()
```

[7]:
```
           President             Born      Age atstart of presidency  \
#
1  George Washington  Feb 22, 1732[a]  57 years, 67 daysApr 30, 1789
2         John Adams  Oct 30, 1735[a]  61 years, 125 daysMar 4, 1797
3   Thomas Jefferson  Apr 13, 1743[a]  57 years, 325 daysMar 4, 1801
4      James Madison  Mar 16, 1751[a]  57 years, 353 daysMar 4, 1809
5       James Monroe     Apr 28, 1758  58 years, 310 daysMar 4, 1817

         Age atend of presidency Post-presidencytimespan          Died  \
#
1   65 years, 10 daysMar 4, 1797       2 years, 285 days  Dec 14, 1799
2  65 years, 125 daysMar 4, 1801      25 years, 122 days   Jul 4, 1826
3  65 years, 325 daysMar 4, 1809      17 years, 122 days   Jul 4, 1826
4  65 years, 353 daysMar 4, 1817      19 years, 116 days  Jun 28, 1836
5  66 years, 310 daysMar 4, 1825       6 years, 122 days   Jul 4, 1831

                  Age Primeiro nome   Sobrenome
#
1  67 years, 295 days        George  Washington
2  90 years, 247 days          John       Adams
3   83 years, 82 days        Thomas   Jefferson
4  85 years, 104 days         James     Madison
5   73 years, 67 days         James      Monroe
```

Agora vamos olhar para a parte do 'Born'

[8]:
```
# Primeiro, vamos nos livrar de tudo que não está no padrão dia, mês, ano
df['Born'] = df['Born'].str.extract('(?P<Data>[\w]{3}\s[\d]{1,2}\,\s[\d]{4})(?:
 ↪\[.*\])?')
df['Born']
```

[8]:
```
#
1     Feb 22, 1732
2     Oct 30, 1735
3     Apr 13, 1743
4     Mar 16, 1751
5     Apr 28, 1758
6     Jul 11, 1767
7     Mar 15, 1767
8      Dec 5, 1782
9      Feb 9, 1773
10    Mar 29, 1790
11     Nov 2, 1795
12    Nov 24, 1784
13     Jan 7, 1800
14    Nov 23, 1804
15    Apr 23, 1791
```

```
16    Feb 12, 1809
17    Dec 29, 1808
18    Apr 27, 1822
19     Oct 4, 1822
20    Nov 19, 1831
21     Oct 5, 1829
22    Mar 18, 1837
23    Aug 20, 1833
24    Mar 18, 1837
25    Jan 29, 1843
26    Oct 27, 1858
27    Sep 15, 1857
28    Dec 28, 1856
29     Nov 2, 1865
30     Jul 4, 1872
31    Aug 10, 1874
32    Jan 30, 1882
33     May 8, 1884
34    Oct 14, 1890
35    May 29, 1917
36    Aug 27, 1908
37     Jan 9, 1913
38    Jul 14, 1913
39     Oct 1, 1924
40     Feb 6, 1911
41    Jun 12, 1924
42    Aug 19, 1946
43     Jul 6, 1946
44     Aug 4, 1961
Name: Born, dtype: str
```

OBS: As funções '.str.' são vetorizadas

[10]:
```
# Perceba que o tipo dessa coluna é "str", mas o Pandas tem funcionalidades␣
↪interessantes para lidar com data/hora

# Vamos transformar a str em data/hora
df['Born'] = pd.to_datetime(df['Born'])
df['Born'].head()
```

[10]:
```
#
1   1732-02-22
2   1735-10-30
3   1743-04-13
4   1751-03-16
5   1758-04-28
Name: Born, dtype: datetime64[us]
```

Isso faria com que o processamento subsequente nos dados relacionados a datas, como conseguir todos os presidentes nascidos em determinado período, se torne muito mais fácil